

SOLIDWORKS Document Manager API (.NET) — Examples (2/4): Code_Example_VBNET-Get_Configuration-Specific_Custom_Properties

SOLIDWORKS API Documentation (converted from CHM)

DimXpert Example (VB.NET)

This example shows how to call interfaces, methods, and properties in the SOLIDWORKS Document Manager API for parts with DimXpert features and annotations.

To learn how to build parts containing DimXpert features and annotations that work with a particular API, open the examples in the **Examples** section in the SOLIDWORKS Document Manager API topic of interest.

```
'-----
' Preconditions:
' 1. Create a part that contains DimXpert features and
'    annotations (see NOTES below).
' 2. Add the SolidWorks.Interop.swdocumentmgr.dll reference
'    to the project:
'    a. Right-click the project in Project Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click:
'        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
' 3. Substitute your_part_with_dimensions with the path
'    to the part you just created.
' 4. Substitute your_license_code with your license code.
' 5. Compile and run this program.
'
' Postconditions: Inspect the Immediate Window.
'
' NOTES:
' You must convert older parts to the latest supported
' version of SOLIDWORKS in order to get the DimXpert CAD identifiers
' for features and annotations using the API. To convert an older
' part to the latest supported version:
' 1. Open the part in a version of SOLIDWORKS that contains this API.
' 2. Save the part, clicking OK in the pop-up dialog to convert the part.
'    NOTE: If you are using a tutorial part, be sure to
'    save it to another name.
' 3. Close the part.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics
Partial Class SolidWorksMacro
    Sub Main()
        Dim classfac As SwDMClassFactory
        Dim docMgr As SwDMAApplication
        Dim swDoc As SwDMDocument5
        Dim e As SwDmDocumentOpenError
        classfac = CreateObject("SwDocumentMgr.SwDMClassFactory")

        ' Substitute your license code here:
        docMgr = classfac.GetApplication("your_license_code")

        Debug.Print("Latest supported file version: " & docMgr.GetLatestSupportedFileVersion)
        Dim fileName As String

        ' Substitute the path to your DimXpert part here:
        fileName = "your_part_with_dimensions"

        swDoc = docMgr.GetDocument(fileName, SwDmDocumentType.swDmDocumentPart, True, e)
        If (swDoc Is Nothing) Then
            MsgBox(fileName & " not found")
            Exit Sub
        End If
        Debug.Print(" Loaded document version: " & swDoc.GetVersion)
```

```

Debug.Print(" Loaded document name: " & fileName)
Debug.Print(" ")
Dim vConfigNames As Object
Dim configName As String
Dim configObj As SwDMConfiguration12
Dim index As Integer
Dim myDimXpertPart As SwMDimXpertPart
Dim vFeatures As Object, myFeature As SwMDimXpertFeature
Dim vAnnotations As Object, myAnnotation As SwMDimXpertAnnotation
Dim myBlockTolerance As SwMDimXpertBlockTolerances
Dim vObject As Object
Dim featureIndex As Integer, annoIndex As Integer, vObjectIndex As Integer
Dim tempFeature As SwMDimXpertFeature, tempFeature1 As SwMDimXpertFeature
Dim endFeature1 As SwMDimXpertFeature, endFeature2 As SwMDimXpertFeature
Dim planeFeature1 As SwMDimXpertPlaneFeature, planeFeature2 As SwMDimXpertPlaneFeature
Dim datum As swDmDimXpertDatum
Dim fosUsage As swDmDimXpertDistanceFosUsage_e
Dim modifier As swDmDimXpertMaterialConditionModifier_e
Dim bltType As swDmDimXpertBlockToleranceType_e
Dim X As Double, Y As Double, Z As Double
Dim i As Double, j As Double, k As Double
Dim longi As Double, longj As Double, longk As Double
Dim width As Double, length As Double, angle As Double, Distance As Double
Dim Tolerance As Double, Value As Double
Dim isMax As Boolean, Enabled As Boolean
Dim feattype As swDmDimXpertFeatureType_e, annotype As swDmDimXpertAnnotationType_e
Dim myGeoTol As SwMDimXpertGeometricTolerance
Dim myDimTol As SwMDimXpertDimensionTolerance
Dim myOriTol As SwMDimXpertOrientationGeoTol
Dim plusTol As Double, minusTol As Double, lowerTol As Double, upperTol As Double
Dim boolstatus As Boolean
vConfigNames = swDoc.ConfigurationManager.GetConfigurationNames()
If Not (IsNothing(vConfigNames)) Then
    For index = LBound(vConfigNames) To UBound(vConfigNames)
        configName = vConfigNames(index)
        configObj = swDoc.ConfigurationManager.GetConfigurationByName(configName)
        If Not configObj Is Nothing Then
            myDimXpertPart = configObj.DimXpertPart
            If Not myDimXpertPart Is Nothing Then
                vFeatures = myDimXpertPart.GetFeatures()
                If Not (IsNothing(vFeatures)) Then
                    Debug.Print(" ")
                    Debug.Print(myDimXpertPart.GetFeatureCount & " DimXpert features:")
                    Debug.Print(" ")
                    For featureIndex = LBound(vFeatures) To UBound(vFeatures)
                        myFeature = vFeatures(featureIndex)
                        Debug.Print(featureIndex & " : " & "Feature name = " & myFeature.Name)
                        feattype = myFeature.type
                        Select Case feattype
                            Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Plane
                                Debug.Print(" Type = Plane")
                                Dim myPlaneFeature As SwMDimXpertPlaneFeature
                                myPlaneFeature = myFeature
                                boolstatus = myPlaneFeature.GetNominalPlane(X, Y, Z, i, j, k)
                                Debug.Print(" NomPlane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " &
Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
                            Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_BestfitPlane
                                Debug.Print(" Type = Best fit plane")
                                Dim myBestFitPlaneFeature As SwMDimXpertBestFitPlaneFeature
                                myBestFitPlaneFeature = myFeature
                                boolstatus = myBestFitPlaneFeature.GetNominalPlane(X, Y, Z, i, j, k)
                                Debug.Print(" NominalPlane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " &
Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
                                Debug.Print(" Number of sub-features: " & myBestFitPlaneFeature.GetSubFeatureCount)
                                If myBestFitPlaneFeature.GetSubFeatureCount > 0 Then
                                    Debug.Print(" Sub-features:")
                                End If
                            End Select
                        End For
                    End For
                End If
            End If
        End If
    End For
End If

```

```

        vObject = myBestFitPlaneFeature.GetSubFeatures
        For vObjectIndex = LBound(vObject) To UBound(vObject)
            tempFeature = vObject(vObjectIndex)
            Debug.Print("        " & tempFeature.Name)
        Next vObjectIndex
    End If
Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Chamfer
    Debug.Print("    Type = Chamfer")
    Dim myChamferFeature As SwDMDimXpertChamferFeature
    myChamferFeature = myFeature
    Debug.Print("        Chamfer angle: " & myChamferFeature.Angle)
    Debug.Print("        Chamfer angle type (swDmDimXpertChamferAngleType_e): " & myChamferFeature.AngleType)
    Debug.Print("        Chamfer type (swDmDimXpertChamferType_e): " & myChamferFeature.ChamferType)
    Debug.Print("        Distance 1: " & myChamferFeature.Distance1)
    Debug.Print("        Distance 2: " & myChamferFeature.Distance2)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_CompoundClosedSlot3D
    Debug.Print("    Type = Compound closed slot 3D")
    Dim myClosedSlotFeature As SwDMDimXpertCompoundClosedSlot3dFeature
    myClosedSlotFeature = myFeature
    tempFeature = myClosedSlotFeature.GetBottomFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Bottom feature: " & tempFeature.Name)
    End If
    endFeature1 = Nothing
    endFeature2 = Nothing
    planeFeature1 = Nothing
    planeFeature2 = Nothing
    boolstatus = myClosedSlotFeature.GetEndFeatures(endFeature1, endFeature2)
    If Not endFeature1 Is Nothing And Not endFeature2 Is Nothing Then
        Debug.Print("        End features: " & endFeature1.Name & ", " & endFeature2.Name)
    End If
    boolstatus = myClosedSlotFeature.GetNominalBottomPlane(X, Y, Z, i, j, k)
    Debug.Print("        Bottom plane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") &
        ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    boolstatus = myClosedSlotFeature.GetNominalClosedSlot(width, length, X, Y, Z, i, j, k, longi, longj,
        longk)
    Debug.Print("        Closed slot(width,length,x,j,z,i,j,k,longi,longj,longk): " & Format(width, "0.0####") &
        ", " & Format(length, "0.0####") & ", " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i,
        "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####") & ", " & Format(longi, "0.0#####") & ", " & Format(longj, "0.0#####") &
        ", " & Format(longk, "0.0#####"))
    boolstatus = myClosedSlotFeature.GetNominalTopPlane(X, Y, Z, i, j, k)
    Debug.Print("        Top plane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " &
        & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    boolstatus = myClosedSlotFeature.GetPlaneFeatures(planeFeature1, planeFeature2)
    If Not planeFeature1 Is Nothing And Not planeFeature2 Is Nothing Then
        Debug.Print("        Plane features: " & planeFeature1.Name & ", " & planeFeature2.Name)
    End If
    Debug.Print("        Closed slot is blind and not through all: " & myClosedSlotFeature.Blind)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_CompoundHole
    Debug.Print("    Type = Compound hole")
    Dim myHole As SwDMDimXpertCompoundHoleFeature
    myHole = myFeature
    Debug.Print("        Hole is blind and not through all: " & myHole.Blind)
    Dim bottomFeature As SwDMDimXpertFeature
    bottomFeature = myHole.GetBottomFeature()
    If Not (bottomFeature Is Nothing) Then
        Debug.Print("        Bottom feature type (swDmDimXpertFeatureType_e) = " & bottomFeature.type)
    End If
    Dim refFeature As SwDMDimXpertFeature
    refFeature = myHole.GetReferenceFeature()
    If Not (refFeature Is Nothing) Then
        Debug.Print("        Reference feature type (swDmDimXpertFeatureType_e) = " & refFeature.type)
    End If
    If myHole.GetSubFeatureCount > 0 Then
        vObject = myHole.GetSubFeatures
        Debug.Print("        Sub-features: ")
        For vObjectIndex = LBound(vObject) To UBound(vObject)
            tempFeature = vObject(vObjectIndex)

```

```

        Debug.Print("        " & tempFeature.Name)
    Next vObjectIndex
End If
Case swDmDimXpertFeatureType_e.swDmDimXpertFeatureType_CompoundNotch
    Debug.Print("    Type = Compound notch")
    Dim myNotchFeature As SwDMDimXpertCompoundNotchFeature
    myNotchFeature = myFeature
    tempFeature = myNotchFeature.GetBottomFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Bottom feature: " & tempFeature.Name)
    End If
    endFeature1 = myNotchFeature.GetEndFeature
    If Not endFeature1 Is Nothing Then
        Debug.Print("        End feature: " & endFeature1.Name)
    End If
    boolstatus = myNotchFeature.GetNominalBottomPlane(X, Y, Z, i, j, k)
    Debug.Print("        Bottom plane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    boolstatus = myNotchFeature.GetNominalNotch(width, length, X, Y, Z, i, j, k, longi, longj, longk)
    Debug.Print("        Notch(width,length,x,j,z,i,j,k,longi,longj,longk): " & Format(width, "0.0####") & ", " & Format(length, "0.0####") & ", " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####") & ", " & Format(longi, "0.0#####") & ", " & Format(longj, "0.0#####") & ", " & Format(longk, "0.0#####"))
    tempFeature = myNotchFeature.GetOpenSideReferenceFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Open side reference feature: " & tempFeature.Name)
    End If
    tempFeature = myNotchFeature.GetTopReferenceFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Top reference feature: " & tempFeature.Name)
    End If
    planeFeature1 = Nothing
    planeFeature2 = Nothing
    boolstatus = myNotchFeature.GetPlaneFeatures(planeFeature1, planeFeature2)
    If Not planeFeature1 Is Nothing And Not planeFeature2 Is Nothing Then
        Debug.Print("        Plane features: " & planeFeature1.Name & ", " & planeFeature2.Name)
    End If
    Debug.Print("        Notch is blind and not through all: " & myNotchFeature.Blind)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeatureType_CompoundWidth
    Debug.Print("    Type = Compound width")
    Dim myWidthFeature As SwDMDimXpertCompoundWidthFeature
    myWidthFeature = myFeature
    boolstatus = myWidthFeature.GetNominalCompoundWidth(width, X, Y, Z, i, j, k)
    Debug.Print("        Compound width(width,x,j,z,i,j,k): " & Format(width, "0.0####") & ", " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    boolstatus = myWidthFeature.GetNominalLongitude(i, j, k)
    Debug.Print("        Nominal longitude(longi, longj, longk): " & Format(longi, "0.0#####") & ", " & Format(longj, "0.0#####") & ", " & Format(longk, "0.0#####"))
    Debug.Print("        Compound width is for a hole and not a pin: " & myWidthFeature.Inner)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeatureType_Cone
    Debug.Print("    Type = Cone")
    Dim myConeFeature As SwDMDimXpertConeFeature
    myConeFeature = myFeature
    boolstatus = myConeFeature.GetNominalBottomPlane(X, Y, Z, i, j, k)
    Debug.Print("        Nominal bottom plane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    boolstatus = myConeFeature.GetNominalCone(angle, X, Y, Z, i, j, k)
    Debug.Print("        Cone(angle,x,j,z,i,j,k): " & Format(angle, "0.0#####") & ", " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    boolstatus = myConeFeature.GetNominalTopPlane(X, Y, Z, i, j, k)
    Debug.Print("        Nominal top plane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    Debug.Print("        Cone is a hole and not a pin: " & myConeFeature.Inner)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeatureType_Cylinder
    Debug.Print("    Type = Cylinder")
    Dim myCylinder As SwDMDimXpertCylinderFeature
    myCylinder = myFeature

```

```

        boolstatus = myCylinder.GetNominalTopPlane(X, Y, Z, i, j, k)
        Debug.Print("    TopPlane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ", " &
Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
        boolstatus = myCylinder.GetNominalCylinder(angle, X, Y, Z, i, j, k)
        Debug.Print("    Cylinder(radius,x,j,z,i,j,k): " & Format(angle, "0.0####") & ", " & Format(X,
"0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " &
Format(k, "0.0#####"))

        boolstatus = myCylinder.GetNominalBottomPlane(X, Y, Z, i, j, k)
        Debug.Print("    BottomPlane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") & ",
" & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
        Dim isThreaded As Boolean, threadDesignation As String, threadDepth As Double
        threadDesignation = Nothing
        boolstatus = myCylinder.GetThread(isThreaded, threadDesignation, threadDepth)
        Debug.Print("    Cylinder is threaded: " & isThreaded)
        If (isThreaded) Then
            Debug.Print("    Cylinder thread designation: " & threadDesignation)
            Debug.Print("    Cylinder thread depth: " & threadDepth)
        End If
        Debug.Print("    Cylinder is a hole and not a pin: " & myCylinder.Inner)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Extrude
    Debug.Print("    Type = Extrude")
    Dim myExtrude As SwDMDimXpertExtrudeFeature
    myExtrude = myFeature
    tempFeature = myExtrude.GetBottomFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Bottom feature: " & tempFeature.Name)
    End If
    tempFeature = myExtrude.GetBottomBlends
    If Not tempFeature Is Nothing Then
        Debug.Print("        Bottom blends: " & tempFeature.Name)
    End If
    tempFeature = myExtrude.GetTopBlends
    If Not tempFeature Is Nothing Then
        Debug.Print("        Top blends: " & tempFeature.Name)
    End If
    tempFeature = myExtrude.GetReferenceFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Reference feature: " & tempFeature.Name)
    End If
    Debug.Print("    Number of sub-features: " & myExtrude.GetSubFeatureCount)
    If myExtrude.GetSubFeatureCount > 0 Then
        Debug.Print("        Sub-features:")
        vObject = myExtrude.GetSubFeatures
        For vObjectIndex = LBound(vObject) To UBound(vObject)
            tempFeature = vObject(vObjectIndex)
            Debug.Print("            " & tempFeature.Name)
        Next vObjectIndex
    End If
    Debug.Print("        Extrude is blind and not through all: " & myExtrude.Blind)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Fillet
    Debug.Print("    Type = Fillet")
    Dim myFillet As SwDMDimXpertFilletFeature
    myFillet = myFeature
    Debug.Print("        Fillet is for a hole and not a pin: " & myFillet.Inner)
    Debug.Print("        Radius = " & myFillet.Radius)
Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectCircle
    Debug.Print("    Type = Intersect circle")
    Dim myICircle As SwDMDimXpertIntersectCircleFeature
    myICircle = myFeature
    tempFeature = myICircle.GetIntersectFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Intersect feature: " & tempFeature.Name)
    End If
    tempFeature = myICircle.GetPlaneFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("        Plane feature: " & tempFeature.Name)
    End If

```

```

        boolstatus = myICircle.GetNominalCircle(angle, X, Y, Z, i, j, k)
        Debug.Print("    Intersect circle(radius,x,y,z,i,j,k): " & Format(angle, "0.0#####") & ", " & Format(X,
"0.0#####") & ", " & Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " &
Format(k, "0.0#####"))

    Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectLine
        Debug.Print("    Type = Intersect line")
        Dim myILine As SwDMDimXpertIntersectLineFeature
        myILine = myFeature
        tempFeature = Nothing
        tempFeature1 = Nothing
        boolstatus = myILine.GetPlaneFeatures(tempFeature, tempFeature1)
        If Not tempFeature Is Nothing And Not tempFeature1 Is Nothing Then
            Debug.Print("    Plane feature 1: " & tempFeature.Name)
            Debug.Print("    Plane feature 2: " & tempFeature1.Name)
        End If
        boolstatus = myILine.GetNominalLine(X, Y, Z, i, j, k)
        Debug.Print("    Intersect line(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") &
", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectPlane
        Debug.Print("    Type = Intersect plane")
        Dim myIPlane As SwDMDimXpertIntersectPlaneFeature
        myIPlane = myFeature
        tempFeature = Nothing
        tempFeature1 = Nothing
        boolstatus = myIPlane.GetFeatures(tempFeature, tempFeature1)
        If Not tempFeature Is Nothing And Not tempFeature1 Is Nothing Then
            Debug.Print("    Feature 1: " & tempFeature.Name)
            Debug.Print("    Feature 2: " & tempFeature1.Name)
        End If
        boolstatus = myIPlane.GetNominalPlane(X, Y, Z, i, j, k)
        Debug.Print("    Intersect plane(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") &
", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_IntersectPoint
        Debug.Print("    Type = Intersect point")
        Dim myIPoint As SwDMDimXpertIntersectPointFeature
        myIPoint = myFeature
        tempFeature = Nothing
        tempFeature1 = Nothing
        boolstatus = myIPoint.GetFeatures(tempFeature, tempFeature1)
        If Not tempFeature Is Nothing And Not tempFeature1 Is Nothing Then
            Debug.Print("    Feature 1: " & tempFeature.Name)
            Debug.Print("    Feature 2: " & tempFeature1.Name)
        End If
        boolstatus = myIPoint.GetNominalPoint(X, Y, Z)
        boolstatus = myIPoint.GetNominalVector(i, j, k)
        Debug.Print("    Intersect point(x,y,z,i,j,k): " & Format(X, "0.0#####") & ", " & Format(Y, "0.0#####") &
", " & Format(Z, "0.0#####") & ", " & Format(i, "0.0#####") & ", " & Format(j, "0.0#####") & ", " & Format(k, "0.0#####"))
    Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Pattern
        Debug.Print("    Type = Pattern")
        Dim myPattern As SwDMDimXpertPatternFeature
        myPattern = myFeature
        Debug.Print("    Number of sub-features: " & myPattern.GetSubFeatureCount)
        If myPattern.GetSubFeatureCount > 0 Then
            Debug.Print("    Sub-features:")
            vObject = myPattern.GetSubFeatures
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                tempFeature = vObject(vObjectIndex)
                Debug.Print("        " & tempFeature.Name)
            Next vObjectIndex
        End If
    Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Sphere
        Debug.Print("    Type = Sphere")
        Dim mySphere As SwDMDimXpertSphereFeature
        mySphere = myFeature
        boolstatus = mySphere.GetNominalSphere(angle, X, Y, Z)
        Debug.Print("    Sphere(radius,x,j,z): " & Format(angle, "0.0#####") & ", " & Format(X, "0.0#####") & ", " &
Format(Y, "0.0#####") & ", " & Format(Z, "0.0#####"))
        Debug.Print("    Sphere is for a hole and not a pin: " & mySphere.Inner)

```

```

        Case swDmDimXpertFeatureType_e.swDmDimXpertFeature_Surface
            Debug.Print(" Type = Surface")
        Case Else
            Debug.Print(" Type = <unknown>")
        End Select
        Debug.Print(" CAD identifier count = " & myFeature.GetCadIdentifierCount)
        If myFeature.GetCadIdentifierCount > 0 Then
            Debug.Print(" CAD Identifiers:")
            vObject = myFeature.GetCadIdentifiers
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                Debug.Print(" " & vObject(vObjectIndex))
            Next vObjectIndex
        End If
        Debug.Print(" Suppressed = " & myFeature.IsSuppressed)
        Debug.Print(" ")
    Next featureIndex
End If
myBlockTolerance = myDimXpertPart.GetBlockTolerances
Debug.Print(" DimXpert block tolerance settings:")
Debug.Print(" ")
Dim linear1 As Double, linear2 As Double, linear3 As Double, angular As Double
Dim linear1Prec As Integer, linear2Prec As Integer, linear3Prec As Integer
bltType = myBlockTolerance.type
Select Case bltType
    Case swDmDimXpertBlockToleranceType_e.swDmDimXpertBlockToleranceType_ASMEInch
        boolstatus = myBlockTolerance.GetToleranceValues(linear1, linear1Prec, linear2, linear2Prec, linear3,
linear3Prec, angular)
        Debug.Print(" ASMEInch Dimension 1 tolerance: " & linear1)
        Debug.Print(" ASMEInch Dimension 1 tolerance precision: " & linear1Prec)
        Debug.Print(" ASMEInch Dimension 2 tolerance: " & linear2)
        Debug.Print(" ASMEInch Dimension 2 tolerance precision: " & linear2Prec)
        Debug.Print(" ASMEInch Dimension 3 tolerance: " & linear3)
        Debug.Print(" ASMEInch Dimension 3 tolerance precision: " & linear3Prec)
        Debug.Print(" ASMEInch Angular tolerance: " & angular)
    Case swDmDimXpertBlockToleranceType_e.swDmDimXpertBlockToleranceType_ISO2768
        Dim ISO2768Type As swDmDimXpertISO2768PartType_e
        boolstatus = myBlockTolerance.GetISO2768PartType(ISO2768Type)
        Debug.Print(" ISO 2768 part type (swDmDimXpertISO2768PartType_e): " & ISO2768Type)
    Case swDmDimXpertBlockToleranceType_e.swDmDimXpertBlockToleranceType_unknown
        Debug.Print(" Block tolerance unknown")
End Select
vAnnotations = myDimXpertPart.GetAnnotations()
If Not (IsNothing(vAnnotations)) Then
    Debug.Print(" ")
    Debug.Print(myDimXpertPart.GetAnnotationCount & " DimXpert annotations:")
    Debug.Print(" ")
    For annoIndex = LBound(vAnnotations) To UBound(vAnnotations)
        myDimTol = Nothing
        myGeoTol = Nothing
        myOriTol = Nothing
        myAnnotation = vAnnotations(annoIndex)
        Debug.Print(annoIndex & " : " & "Annotation name = " & myAnnotation.Name)
        annotype = myAnnotation.type
        Select Case annotype
            Case swDmDimXpertAnnotationType_e.swDmDimXpertDatum
                Debug.Print(" Type = Datum")
                Dim myDatum As swDmDimXpertDatum
                myDatum = myAnnotation
                Debug.Print(" ID = " & myDatum.Identifier)
            Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_AngleBetween
                Debug.Print(" Type = Angle-between dimension tolerance")
                Dim myAngBetDimTol As swDmDimXpertAngleBetweenDimTol
                myAngBetDimTol = myAnnotation
                boolstatus = myAngBetDimTol.GetDirectionVector(i, j, k)
                Debug.Print(" DirectionVector(i,j,k): " & i & ", " & j & ", " & k)
                Debug.Print(" Feature of origin: " & myAngBetDimTol.OriginFeature.Name)
            End Select
        End If
    Next annoIndex
End If

```

```

        Debug.Print("    Calculate supplement angle? " & myAngBetDimTol.Supplement)
        myDimTol = myAnnotation
    Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_ChamferDimension
        Debug.Print("    Type = Chamfer dimension tolerance")
        myDimTol = myAnnotation
        Dim myChamferDimTol As SwMDMDimXpertChamferDimTol
        myChamferDimTol = myAnnotation
        Debug.Print("    Chamfer dimension type (swDmDimXpertChamferDimensionType_e): " &
myChamferDimTol.ChamferType)

    Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CompositeDistanceBetween
        Debug.Print("    Type = Composite distance-between dimension tolerance")
        myDimTol = myAnnotation
        Dim myCompDistBetDimTol As SwMDMDimXpertCompositeDistanceBetweenDimTol
        myCompDistBetDimTol = myAnnotation
        boolstatus = myCompDistBetDimTol.GetDirectionVector(i, j, k)
        Debug.Print("    Direction vector(i,j,k): " & i & ", " & j & ", " & k)
        tempFeature1 = Nothing
        boolstatus = myCompDistBetDimTol.GetFeature(tempFeature1, fosUsage)
        If Not tempFeature1 Is Nothing Then
            Debug.Print("    Composite distance-between dimension tolerance feature: " & tempFeature1.Name)
            Debug.Print("    Feature of size usage (swDmDimXpertDistanceFosUsage_e): " & fosUsage)
        End If
        boolstatus = myCompDistBetDimTol.GetIntraFeature(tempFeature1, fosUsage)
        If Not tempFeature1 Is Nothing Then
            Debug.Print("    Feature-locating feature: " & tempFeature1.Name)
            Debug.Print("    Feature of size usage for feature-locating(swDmDimXpertDistanceFosUsage_e): " &
fosUsage)
        End If
        boolstatus = myCompDistBetDimTol.GetOriginFeature(tempFeature1, fosUsage)
        If Not tempFeature1 Is Nothing Then
            Debug.Print("    Feature of origin for this dimension tolerance: " & tempFeature1.Name)
            Debug.Print("    Feature of size usage (swDmDimXpertDistanceFosUsage_e): " & fosUsage)
        End If
        Debug.Print("    Type of dimension tolerance for feature-locating
(swDmDimXpertDimensionToleranceType_e): " & myCompDistBetDimTol.DimensionTypeIntraFeature)
        Debug.Print("    Tolerance for feature-locating: " & myCompDistBetDimTol.GetNominalValueIntraFeature)
        boolstatus = myCompDistBetDimTol.GetPlusAndMinusToleranceIntraFeature(plusTol, minusTol)
        Debug.Print("    Plus / Minus limit for feature-locating: " & plusTol & " / " & minusTol)
        boolstatus = myCompDistBetDimTol.GetUpperAndLowerLimitIntraFeature(upperTol, lowerTol)
        Debug.Print("    Upper / Lower limit for feature-locating: " & upperTol & " / " & lowerTol)
        Debug.Print("    Block tolerance precision for feature-locating: " &
myCompDistBetDimTol.BlockToleranceDecimalPlacesIntraFeature)
        Debug.Print("    Limits and fits code for feature-locating: " &
myCompDistBetDimTol.LimitsAndFitsCodeIntraFeature)

    Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_ConeAngle
        Debug.Print("    Type = Cone angle dimension tolerance")
        myDimTol = myAnnotation

    Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CounterBore
        Debug.Print("    Type = Counterbore dimension tolerance")
        myDimTol = myAnnotation
        Dim myCounterBoreDimTol As SwMDMDimXpertCounterBoreDimTol
        myCounterBoreDimTol = myAnnotation
        tempFeature = myCounterBoreDimTol.ReferenceFeature
        If Not tempFeature Is Nothing Then
            Debug.Print("    Reference Feature for this dimension tolerance: " & tempFeature.Name)
        End If

    Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CounterSinkAngle
        Debug.Print("    Type = Countersink angle dimension tolerance")
        myDimTol = myAnnotation
        Dim myCounterSinkAngleDimTol As SwMDMDimXpertCounterSinkAngleDimTol
        myCounterSinkAngleDimTol = myAnnotation
        tempFeature = myCounterSinkAngleDimTol.ReferenceFeature
        If Not tempFeature Is Nothing Then
            Debug.Print("    Reference Feature for this dimension tolerance: " & tempFeature.Name)
        End If

    Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_CounterSinkDiameter
        Debug.Print("    Type = Countersink diameter dimension tolerance")

```

```

myDimTol = myAnnotation
Dim myCounterSinkDiameterDimTol As SwDMDimXpertCounterSinkDiameterDimTol
myCounterSinkDiameterDimTol = myAnnotation
tempFeature = myCounterSinkDiameterDimTol.ReferenceFeature
If Not tempFeature Is Nothing Then
    Debug.Print("    Reference Feature for this dimension tolerance: " & tempFeature.Name)
End If
Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Depth
    Debug.Print("    Type = Depth dimension tolerance")
    myDimTol = myAnnotation
    Dim myDepthDimTol As SwDMDimXpertDepthDimTol
    myDepthDimTol = myAnnotation
    Debug.Print("    Tolerance is for thread depth: " & myDepthDimTol.IsThreadDepth)
    tempFeature = myDepthDimTol.ReferenceFeature
    If Not tempFeature Is Nothing Then
        Debug.Print("    Reference Feature for this dimension tolerance: " & tempFeature.Name)
    End If
Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Diameter
    Debug.Print("    Type = Diameter dimension tolerance")
    myDimTol = myAnnotation
    Dim myDiameterDimTol As SwDMDimXpertDiameterDimTol
    myDiameterDimTol = myAnnotation
    Debug.Print("    Pattern treatment type (swDmDimXpertPatternTreatmentType_e): " &
myDiameterDimTol.PatternTreatment)
Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_DistanceBetween
    Debug.Print("    Type = Distance-between dimension tolerance")
    myDimTol = myAnnotation
    Dim myDistBetDimTol As SwDMDimXpertDistanceBetweenDimTol
    myDistBetDimTol = myAnnotation
    boolstatus = myDistBetDimTol.GetDirectionVector(i, j, k)
    Debug.Print("    Direction vector(i,j,k): " & i & ", " & j & ", " & k)
    tempFeature1 = Nothing
    boolstatus = myDistBetDimTol.GetFeature(tempFeature1, fosUsage)
    If Not tempFeature1 Is Nothing Then
        Debug.Print("    Distance-between dimension tolerance feature: " & tempFeature1.Name)
        Debug.Print("    Feature of size usage (swDmDimXpertDistanceFosUsage_e): " & fosUsage)
    End If
    boolstatus = myDistBetDimTol.GetOriginFeature(tempFeature1, fosUsage)
    If Not tempFeature1 Is Nothing Then
        Debug.Print("    Feature of origin for this dimension tolerance: " & tempFeature1.Name)
        Debug.Print("    Feature of size usage(swDmDimXpertDistanceFosUsage_e): " & fosUsage)
    End If
    Debug.Print("    Pattern treatment type (swDmDimXpertPatternTreatmentType_e): " &
myDistBetDimTol.PatternTreatment)
Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Length
    Debug.Print("    Type = Length dimension tolerance")
    myDimTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Radius
    Debug.Print("    Type = Radius dimension tolerance")
    myDimTol = myAnnotation
    Dim myRadiusDimTol As SwDMDimXpertRadiusDimTol
    myRadiusDimTol = myAnnotation
    Debug.Print("    Pattern treatment type (swDmDimXpertPatternTreatmentType_e): " &
myRadiusDimTol.PatternTreatment)
Case swDmDimXpertAnnotationType_e.swDmDimXpertDimTol_Width
    Debug.Print("    Type = Width dimension tolerance")
    myDimTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Angularity
    Debug.Print("    " & annoIndex & " : Type = Angularity geometric tolerance")
    myGeoTol = myAnnotation
    myOriTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Circularity
    Debug.Print("    Type = Circularity geometric tolerance")
    myGeoTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CircularRunout
    Debug.Print("    Type = Circular runout geometric tolerance")

```

```

        myGeoTol = myAnnotation
    Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CompositeLineProfile
        Debug.Print(" Type = Composite line profile geometric tolerance")
        myGeoTol = myAnnotation
        Dim myCompLineProfileGeoTol As SwDMDimXpertCompositeLineProfileGeoTol
        myCompLineProfileGeoTol = myAnnotation
        boolstatus = myCompLineProfileGeoTol.GetPlanarZoneVector(i, j, k)
        Debug.Print(" Planar zone vector(i,j,k): " & i & ", " & j & ", " & k)
        Debug.Print(" Outer tolerance value: " & myCompLineProfileGeoTol.OuterToleranceValue)
        Debug.Print(" Outer tolerance value in the lower tier: " &
myCompLineProfileGeoTol.OuterToleranceValueLowerTier)
        Debug.Print(" Primary datum repeated in lower tier: " & myCompLineProfileGeoTol.PrimaryInLowerTier)
        Debug.Print(" Secondary datum repeated in lower tier: " &
myCompLineProfileGeoTol.SecondaryInLowerTier)
        Debug.Print(" Tertiary datum repeated in lower tier: " & myCompLineProfileGeoTol.TertiaryInLowerTier)
        Debug.Print(" Tolerance in lower tier: " & myCompLineProfileGeoTol.ToleranceLowerTier)
    Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CompositePosition
        Debug.Print(" Type = Composite position geometric tolerance")
        myGeoTol = myAnnotation
        Dim myCompPosGeoTol As SwDMDimXpertCompositePositionGeoTol
        myCompPosGeoTol = myAnnotation
        boolstatus = myCompPosGeoTol.GetMaxTolerance(isMax, Tolerance)
        Debug.Print(" Maximum tolerance is set: " & isMax)
        If isMax Then
            Debug.Print(" Maximum tolerance: " & Tolerance)
        End If
        boolstatus = myCompPosGeoTol.GetMaxToleranceLowerTier(isMax, Tolerance)
        Debug.Print(" Maximum tolerance is set in lower tier: " & isMax)
        If isMax Then
            Debug.Print(" Maximum tolerance in lower tier: " & Tolerance)
        End If
        boolstatus = myCompPosGeoTol.GetPlanarZoneVector(i, j, k)
        Debug.Print(" Planar zone vector(i,j,k): " & i & ", " & j & ", " & k)
        boolstatus = myCompPosGeoTol.GetProjectedZone(Enabled, Value)
        Debug.Print(" Projected zone is in force: " & Enabled)
        If Enabled Then
            Debug.Print("Value: " & Value)
        End If
        Debug.Print(" Material condition modifier (swDmDimXpertMaterialConditionModifier_e): " &
myCompPosGeoTol.Modifier)
        Debug.Print(" Material condition modifier in lower tier (swDmDimXpertMaterialConditionModifier_e): " &
& myCompPosGeoTol.ModifierLowerTier)
        Debug.Print(" Primary datum repeated in lower tier: " & myCompPosGeoTol.PrimaryInLowerTier)
        Debug.Print(" Secondary datum repeated in lower tier: " & myCompPosGeoTol.SecondaryInLowerTier)
        Debug.Print(" Tertiary datum repeated in lower tier: " & myCompPosGeoTol.TertiaryInLowerTier)
        Debug.Print(" Tolerance in lower tier: " & myCompPosGeoTol.ToleranceLowerTier)
        Debug.Print(" Position zone type (swDmDimXpertPositionZoneType_e): " & myCompPosGeoTol.ZoneType)
    Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_CompositeSurfaceProfile
        Debug.Print(" Type = Composite surface profile geometric tolerance")
        myGeoTol = myAnnotation
        Dim myCompSurfProfGeoTol As SwDMDimXpertCompositeSurfaceProfileGeoTol
        myCompSurfProfGeoTol = myAnnotation
        Debug.Print(" Outer tolerance: " & myCompSurfProfGeoTol.OuterToleranceValue)
        Debug.Print(" Outer tolerance in lower tier: " & myCompSurfProfGeoTol.OuterToleranceValueLowerTier)
        Debug.Print(" Primary datum repeated in lower tier: " & myCompSurfProfGeoTol.PrimaryInLowerTier)
        Debug.Print(" Secondary datum repeated in lower tier: " & myCompSurfProfGeoTol.SecondaryInLowerTier)
        Debug.Print(" Tertiary datum repeated in lower tier: " & myCompSurfProfGeoTol.TertiaryInLowerTier)
        Debug.Print(" Tolerance in lower tier: " & myCompSurfProfGeoTol.ToleranceLowerTier)
    Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Concentricity
        Debug.Print(" Type = Concentricity geometric tolerance")
        myGeoTol = myAnnotation
        Dim myConcentricityGeoTol As SwDMDimXpertConcentricityGeoTol
        myConcentricityGeoTol = myAnnotation
        boolstatus = myConcentricityGeoTol.GetMaxTolerance(isMax, Tolerance)
        Debug.Print(" Maximum tolerance is set: " & isMax)
        If isMax Then
            Debug.Print(" Maximum tolerance: " & Tolerance)

```

```

End If
Debug.Print("    Material condition modifier (swDmDimXpertMaterialConditionModifier_e): " &
myConcentricityGeoTol.Modifier)

Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Cylindricity
    Debug.Print("    Type = Cylindricity geometric tolerance")
    myGeoTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Flatness
    Debug.Print("    Type = Flatness geometric tolerance")
    myGeoTol = myAnnotation
    Dim myFlatnessGeoTol As SwDMDimXpertFlatnessGeoTol
    myFlatnessGeoTol = myAnnotation
    boolstatus = myFlatnessGeoTol.GetPerUnitArea(Enabled, length, width, i, j, k)
    Debug.Print("        Per unit area(enabled,width,length,i,j,k): " & Enabled & ", " & Format(length,
"0.0####") & ", " & Format(width, "0.0####") & ", " & Format(i, "0.0####") & ", " & Format(j, "0.0####") & ", " & Format(k, "0.0####"))
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_LineProfile
    Debug.Print("    Type = Line profile geometric tolerance")
    myGeoTol = myAnnotation
    Dim myLineProfileGeoTol As SwDMDimXpertLineProfileGeoTol
    myLineProfileGeoTol = myAnnotation
    boolstatus = myLineProfileGeoTol.GetPlanarZoneVector(i, j, k)
    Debug.Print("        Planar zone vector(i,j,k): " & i & ", " & j & ", " & k)
    Debug.Print("        Outer tolerance value: " & myLineProfileGeoTol.OuterToleranceValue)
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Parallelism
    Debug.Print("    Type = Parallelism geometric tolerance")
    myGeoTol = myAnnotation
    myOriTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Perpendicularity
    Debug.Print("    Type = Perpendicularity geometric tolerance")
    myGeoTol = myAnnotation
    myOriTol = myAnnotation
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Position
    Debug.Print("    Type = Position geometric tolerance")
    myGeoTol = myAnnotation
    Dim myPosGeoTol As SwDMDimXpertPositionGeoTol
    myPosGeoTol = myAnnotation
    boolstatus = myPosGeoTol.GetMaxTolerance(isMax, Tolerance)
    Debug.Print("        Maximum tolerance is set: " & isMax)
    If isMax Then
        Debug.Print("        Maximum tolerance: " & Tolerance)
    End If
    boolstatus = myPosGeoTol.GetPlanarZoneVector(i, j, k)
    Debug.Print("        Planar zone vector(i,j,k): " & i & ", " & j & ", " & k)
    boolstatus = myPosGeoTol.GetProjectedZone(Enabled, Value)
    Debug.Print("        Projected zone is in force: " & Enabled)
    If Enabled Then
        Debug.Print("        Value: " & Value)
    End If
    Debug.Print("    Material condition modifier (swDmDimXpertMaterialConditionModifier_e): " &
myPosGeoTol.Modifier)

    Debug.Print("        Position zone type (swDmDimXpertPositionZoneType_e): " & myPosGeoTol.ZoneType)
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Straightness
    Debug.Print("    Type = Straightness geometric tolerance")
    myGeoTol = myAnnotation
    Dim myStraightnessGeoTol As SwDMDimXpertStraightnessGeoTol
    myStraightnessGeoTol = myAnnotation
    boolstatus = myStraightnessGeoTol.GetMaxTolerance(isMax, Tolerance)
    Debug.Print("        Maximum tolerance is set: " & isMax)
    If isMax Then
        Debug.Print("        Maximum tolerance: " & Tolerance)
    End If
    boolstatus = myStraightnessGeoTol.GetPerUnitLength(Enabled, Distance)
    Debug.Print("        Per unit length(enabled,distance): " & Enabled & ", " & Format(Distance, "0.0####"))
    boolstatus = myStraightnessGeoTol.GetPlanarZoneVector(i, j, k)
    Debug.Print("        Planar zone vector(i,j,k): " & i & ", " & j & ", " & k)
    Debug.Print("    Material condition modifier (swDmDimXpertMaterialConditionModifier_e): " &
myStraightnessGeoTol.Modifier)

```

```

myStraightnessGeoTol.ZoneType)

        Debug.Print("      Position zone type (swDmDimXpertStraightnessZoneType_e): " &

Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_SurfaceProfile
    Debug.Print("  Type = SurfaceProfile geometric tolerance")
    myGeoTol = myAnnotation
    Dim mySurfProfGeoTol As SwDMDimXpertSurfaceProfileGeoTol
    mySurfProfGeoTol = myAnnotation
    Debug.Print("      Outer tolerance: " & mySurfProfGeoTol.OuterToleranceValue)
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Symmetry
    Debug.Print("  Type = Symmetry geometric tolerance")
    myGeoTol = myAnnotation
    Dim mySymmetryGeoTol As SwDMDimXpertSymmetryGeoTol
    mySymmetryGeoTol = myAnnotation
    boolstatus = mySymmetryGeoTol.GetMaxTolerance(isMax, Tolerance)
    Debug.Print("      Maximum tolerance is set: " & isMax)
    Debug.Print("      Maximum tolerance: " & Tolerance)
    boolstatus = mySymmetryGeoTol.GetZoneDirection(i, j, k)
    Debug.Print("      Zone direction(i,j,k): " & i & ", " & j & ", " & k)
    modifier = mySymmetryGeoTol.Modifier
    Debug.Print("      Modifier (swDmDimXpertMaterialConditionModifier_e): " & modifier)
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_Tangency
    Debug.Print("  Type = Tangency geometric tolerance")
    Dim myTangencyGeoTol As SwDMDimXpertTangencyGeoTol
    myTangencyGeoTol = myAnnotation
    tempFeature = Nothing
    boolstatus = myTangencyGeoTol.GetOriginFeature(tempFeature)
    If Not tempFeature Is Nothing Then
        Debug.Print("      Feature of origin for this geometric tolerance: " & tempFeature.Name)
    End If
Case swDmDimXpertAnnotationType_e.swDmDimXpertGeoTol_TotalRunout
    Debug.Print("  Type = Total runout geometric tolerance")
    myGeoTol = myAnnotation
Case Else
    Debug.Print("  Type = <unknown>")
End Select
' Annotation Information
Debug.Print("  Annotation Information:")
Debug.Print("      CAD identifier count = " & myAnnotation.GetCadIdentifierCount)
If myAnnotation.GetCadIdentifierCount > 0 Then
    Debug.Print("      CAD Identifiers:")
    vObject = myAnnotation.GetCadIdentifiers
    For vObjectIndex = LBound(vObject) To UBound(vObject)
        Debug.Print("          " & vObject(vObjectIndex))
    Next vObjectIndex
End If
Debug.Print("      Free state = " & myAnnotation.IsFreeState)
Debug.Print("      Statistical = " & myAnnotation.IsStatistical)
Debug.Print("      Suppressed = " & myAnnotation.IsSuppressed)
Debug.Print("      To be inspected = " & myAnnotation.IsToBeInspected)
If (myAnnotation.Feature Is Nothing) Then
    Debug.Print("      Is not associated with a feature")
Else
    Debug.Print("      Is associated with a feature")
End If
' Geometric Tolerance Information for Orientation Dimensions
If Not myOriTol Is Nothing Then
    Debug.Print("  Orientation Geometric Tolerance Information:")
    boolstatus = myOriTol.GetMaxTolerance(isMax, Tolerance)
    Debug.Print("      Maximum tolerance is set: " & isMax)
    If isMax Then
        Debug.Print("      Maximum tolerance: " & Tolerance)
    End If
    boolstatus = myOriTol.GetPlanarZoneVector(i, j, k)
    Debug.Print("      Planar zone vector(i,j,k): " & i & ", " & j & ", " & k)
    boolstatus = myOriTol.GetProjectedZone(Enabled, Value)
    Debug.Print("      Projected zone is in force: " & Enabled)

```

```

        If Enabled Then
            Debug.Print("Value: " & Value)
        End If
        Debug.Print("    Material condition modifier (swDmDimXpertMaterialConditionModifier_e): " &
myOriTol.Modifier)
        Debug.Print("    Position zone type (swDmDimXpertOrientationZoneType_e): " & myOriTol.ZoneType)
    End If
    ' Geometric Tolerance Information
    If Not myGeoTol Is Nothing Then
        Debug.Print("    Geometric Tolerance Information:")
        Debug.Print("        Number of primary datums: " & myGeoTol.GetPrimaryDatumCount)
        If myGeoTol.GetPrimaryDatumCount > 0 Then
            Debug.Print("            Primary Datum modifiers:")
            vObject = myGeoTol.GetPrimaryDatumModifiers
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                modifier = vObject(vObjectIndex)
                Debug.Print("                Primary datum modifier (swDmDimXpertMaterialConditionModifier_e): " & modifier)
            Next vObjectIndex
            Debug.Print("            Primary datums:")
            vObject = myGeoTol.GetPrimaryDatums
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                datum = vObject(vObjectIndex)
                Debug.Print("                Primary datum : " & datum.Identifier)
            Next vObjectIndex
        End If
        Debug.Print("        Number of secondary datums: " & myGeoTol.GetSecondaryDatumCount)
        If myGeoTol.GetSecondaryDatumCount > 0 Then
            Debug.Print("            Secondary Datum modifiers:")
            vObject = myGeoTol.GetSecondaryDatumModifiers
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                modifier = vObject(vObjectIndex)
                Debug.Print("                Secondary datum modifier (swDmDimXpertMaterialConditionModifier_e): " &
modifier)
            Next vObjectIndex
            Debug.Print("            Secondary datums:")
            vObject = myGeoTol.GetSecondaryDatums
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                datum = vObject(vObjectIndex)
                Debug.Print("                Secondary datum : " & datum.Identifier)
            Next vObjectIndex
        End If
        Debug.Print("        Number of tertiary datums: " & myGeoTol.GetTertiaryDatumCount)
        If myGeoTol.GetTertiaryDatumCount > 0 Then
            Debug.Print("            Tertiary Datum modifiers:")
            vObject = myGeoTol.GetTertiaryDatumModifiers
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                modifier = vObject(vObjectIndex)
                Debug.Print("                Tertiary datum modifier (swDmDimXpertMaterialConditionModifier_e): " & modifier)
            Next vObjectIndex
            Debug.Print("            Tertiary datums:")
            vObject = myGeoTol.GetTertiaryDatums
            For vObjectIndex = LBound(vObject) To UBound(vObject)
                datum = vObject(vObjectIndex)
                Debug.Print("                Tertiary datum : " & datum.Identifier)
            Next vObjectIndex
        End If
        Debug.Print("        Geometric Tolerance = " & myGeoTol.Tolerance)
    End If
    ' Dimension Tolerance Information
    If Not myDimTol Is Nothing Then
        Debug.Print("    Dimension Tolerance Information:")
        Debug.Print("        Dimension tolerance type (swDmDimXpertDimensionToleranceType_e)= " &
myDimTol.DimensionType)
        Debug.Print("        Dimension tolerance = " & myDimTol.GetNominalValue())
        boolstatus = myDimTol.GetPlusAndMinusTolerance(plusTol, minusTol)
        Debug.Print("        Plus / Minus " & plusTol & " / " & minusTol)
    End If

```

```

        boolstatus = myDimTol.GetUpperAndLowerLimit(upperTol, lowerTol)
        Debug.Print("      Upper / Lower  " & upperTol & " / " & lowerTol)
        Debug.Print("      Block tolerance precision = " & myDimTol.BlockToleranceDecimalPlaces)
        Debug.Print("      Limits and fits code = " & myDimTol.LimitsAndFitsCode)
    End If
    Debug.Print(" ")
Next annoIndex
End If
End If
End If
Next index
End If
End Sub
Public swApp As SldWorks
End Class

```

Get, Add, Change, And Delete Cut-List Custom Properties Example (C#)

This example shows how to get cut-list items and their custom properties and how to add, modify, and delete a cut-list custom property.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
//    and ensure that the required DLLs have been registered.
// 2. Copy and paste this class into a C# console application in
//    Microsoft Visual Studio.
// 3. Ensure that the specified part exists or point to another
//    document that contains a cut list with custom properties).
// 4. Add the SolidWorks.Interop.swdocumentmgr.dll reference to
//    the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
//    e. Click Add.
//    f. Click Close.
// 5. Substitute your_license_code with your SOLIDWORKS
//    Document Manager license code.
// 6. Compile and run this program in Debug mode.
//
// Postconditions: Inspect the Output Window and the code to see how
// the API is used.
//-----

using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Text;
using SolidWorks.Interop.swdocumentmgr;

class Program
{
    static void Main(string[] args)
    {
        SwDMClassFactory swClassFact = default(SwDMClassFactory);
        SwDMAApplication swDocMgr = default(SwDMAApplication);
        SwDMDocument10 swDocument10 = default(SwDMDocument10);
        SwDMDocument13 swDocument13 = default(SwDMDocument13);

        SwDMConfiguration16 config = default(SwDMConfiguration16);
        SwDMConfigurationMgr SwDMConfigurationMgr = default(SwDMConfigurationMgr);

        string sDocFileName = null;
        SwDmDocumentType nDocType = default(SwDmDocumentType);
        SwDmDocumentOpenError nRetVal = default(SwDmDocumentOpenError);
        string sLicenseKey = null;

        sLicenseKey = "your_license_code";
        // Specify your SOLIDWORKS Document Manager license key
        sDocFileName = ("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\weldments\\weldment_box2.sldprt");
        // Specify your model document
        nDocType = SwDmDocumentType.swDmDocumentPart;
        swClassFact = new SwDMClassFactory();

        swDocMgr = (SwDMAApplication)swClassFact.GetApplication(sLicenseKey);
        swDocument10 = (SwDMDocument10)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);
        swDocument13 = (SwDMDocument13)swDocument10;

        Debug.Print("Last Update Stamp: " + swDocument13.GetLastUpdateTimeStamp());

        SwDMConfigurationMgr = swDocument13.ConfigurationManager;
        config = (SwDMConfiguration16)SwDMConfigurationMgr.GetConfigurationByName("Default");

        object[] vCutListItems = null;
        vCutListItems = (object[])config.GetCutListItems();

        SwDMCutListItem3 Cutlist = default(SwDMCutListItem3);
        int I = 0;
        SwDmCustomInfoType nType = 0;
        string nLink = null;
        int J = 0;
        object[] vPropNames = null;

        Debug.Print("GET CUT-LIST ITEM");

        for (I = 0; I <= vCutListItems.GetUpperBound(0); I++)
        {
            Cutlist = (SwDMCutListItem3)vCutListItems[I];
            Debug.Print("Name : " + Cutlist.Name);
            Debug.Print(" Quantity : " + Cutlist.Quantity);
            vPropNames = (object[])Cutlist.GetCustomPropertyNames();

            if (!((vPropNames == null)))
            {
                Debug.Print(" GET CUSTOM PROPERTIES");
                for (J = 0; J <= vPropNames.GetUpperBound(0); J++)
                {

```

```

        Debug.Print(" Property Name : " + vPropNames[J]);
        Debug.Print(" Property Value : " + Cutlist.GetCustomPropertyValue2((string)vPropNames[J], out nType, out nLink));
        Debug.Print(" Type : " + nType);
        Debug.Print(" Link : " + nLink);
    }
}

Debug.Print("_____");
}

Cutlist = (SwDMCutListItem3)vCutListItems[0];
Debug.Print("ADD CUSTOM PROPERTY CALLED Testing1");
Debug.Print(" Custom Property added? " + Cutlist.AddCustomProperty("Testing1", SwDmCustomInfoType.swDmCustomInfoText, "Verify1"));
Debug.Print(" GET CUSTOM PROPERTIES");

vPropNames = (object[])Cutlist.GetCustomPropertyNames();

for (J = 0; J <= vPropNames.GetUpperBound(0); J++)
{
    Debug.Print(" Property Name : " + vPropNames[J]);
    Debug.Print(" Property Value : " + Cutlist.GetCustomPropertyValue2((string)vPropNames[J], out nType, out nLink));
    Debug.Print(" Type : " + nType);
    Debug.Print(" Link : " + nLink);
}

Debug.Print("_____");
Debug.Print("SET NEW CUSTOM PROPERTY VALUE FOR Testing1");
Debug.Print(" Property Value Before Setting: " + Cutlist.GetCustomPropertyValue2("Testing1", out nType, out nLink));
Debug.Print(" New Value Set? " + Cutlist.SetCustomProperty("Testing1", "Verify3"));
Debug.Print(" Property Value After Setting : " + Cutlist.GetCustomPropertyValue2("Testing1", out nType, out nLink));
Debug.Print(" GET CUSTOM PROPERTIES");

vPropNames = (object[])Cutlist.GetCustomPropertyNames();

for (J = 0; J <= vPropNames.GetUpperBound(0); J++)
{
    Debug.Print(" Property Name : " + vPropNames[J]);
    Debug.Print(" Property Value : " + Cutlist.GetCustomPropertyValue2((string)vPropNames[J], out nType, out nLink));
    Debug.Print(" Type : " + nType);
    Debug.Print(" Link : " + nLink);
}

Debug.Print("_____");
Debug.Print("DELETE CUSTOM PROPERTY Testing1");
Debug.Print(" Delete Property Value? " + Cutlist.DeleteCustomProperty("Testing1"));

vPropNames = (object[])Cutlist.GetCustomPropertyNames();
if (!(vPropNames == null))
{
    Debug.Print(" GET CUSTOM PROPERTIES");
    for (J = 0; J <= vPropNames.GetUpperBound(0); J++)
    {
        Debug.Print(" Property Name : " + vPropNames[J]);
        Debug.Print(" Property Value : " + Cutlist.GetCustomPropertyValue2((string)vPropNames[J], out nType, out nLink));
        Debug.Print(" Type : " + nType);
        Debug.Print(" Link : " + nLink);
    }
}

Debug.Print("_____");

//swDocument10.Save()

swDocument10.CloseDoc();
}
}

```

Get, Add, Change, And Delete Cut-List Custom Properties Example (VB.NET)

This example shows how to get cut-list items and their custom properties and how to add, modify, and delete a cut-list custom property.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
'    Getting Started topic and ensure that the
'    required DLLs have been registered.
' 2. Copy and paste this module into a VB.NET console
'    application in Microsoft Visual Studio.
' 3. Ensure that the specified part exists or point
'    to another document that contains a cut list with
'    custom properties).
' 4. Add the SolidWorks.Interop.swdocumentmgr.dll reference
'    to the project:
'    a. Right-click the solution in Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
'    e. Click Add.
'    f. Click Close.
' 5. Substitute your_license_code with your SOLIDWORKS
'    Document Manager license key.
' 6. Compile and run this program in Debug mode.
'
' Postconditions: Inspect the Output Window and the code to see
' how the API is used.
'-----

```

```
Imports SolidWorks.Interop.swdocumentmgr
```

```
Module Module1
```

```
Sub main()
```

```
Dim swClassFact As SwDMClassFactory
Dim swDocMgr As SwDMAApplication
Dim swDocument10 As SwDMDocument10
Dim swDocument13 As SwDMDocument13
```

```
Dim swDMConfigurationMgr As SwDMConfigurationMgr
Dim config As SwDMConfiguration16
```

```
Dim sDocFileName As String
Dim nDocType As SwDmDocumentType
Dim nRetVal As SwDmDocumentOpenError
Dim sLicenseKey As String
```

```
sLicenseKey = "your_license_code" ' Specify your SOLIDWORKS Document Manager license key
sDocFileName = ("C:\Users\Public\Documents\SOLIDWORKS 2019\samples\tutorial\weldments\weldment_box2.sldprt") ' Specify your
```

```
model document
```

```
nDocType = SwDmDocumentType.swDmDocumentPart
swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
```

```
swDocMgr = swClassFact.GetApplication(sLicenseKey)
swDocument10 = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal)
swDocument13 = swDocument10
```

```
Debug.Print("Last Update Stamp: " & swDocument13.GetLastUpdateTimeStamp)
```

```
swDMConfigurationMgr = swDocument13.ConfigurationManager
config = swDMConfigurationMgr.GetConfigurationByName("Default")
```

```
Dim vCutListItems As Object
vCutListItems = config.GetCutListItems
```

```
Dim Cutlist As SwDMCutListItem3
Dim I As Integer
Dim nType As Integer
Dim nLink As String
Dim J As Integer
Dim vPropNames As Object
```

```
Debug.Print("GET CUT-LIST ITEM")
```

```
For I = 0 To UBound(vCutListItems)
    Cutlist = vCutListItems(I)
    Debug.Print("Name : " & Cutlist.Name)
    Debug.Print("    Quantity : " & Cutlist.Quantity)
    vPropNames = Cutlist.GetCustomPropertyNames
```

```
    If Not (IsNothing(vPropNames)) Then
        Debug.Print("    GET CUSTOM PROPERTIES")
        For J = 0 To UBound(vPropNames)
            Debug.Print("        Property Name : " & vPropNames(J))
            Debug.Print("        Property Value : " & Cutlist.GetCustomPropertyValue2(vPropNames(J), nType, nLink))
            Debug.Print("        Type : " & nType)
            Debug.Print("        Link : " & nLink)
        Next
    End If

```

```

        Debug.Print("_____")

Next

Cutlist = vCutListItems(0)
Debug.Print("ADD CUSTOM PROPERTY CALLED Testing1")
Debug.Print("    Custom Property added? " & Cutlist.AddCustomProperty("Testing1", SwDmCustomInfoType.swDmCustomInfoText, "Verify1"))
Debug.Print("    GET CUSTOM PROPERTIES")

vPropNames = Cutlist.GetCustomPropertyNames

For J = 0 To UBound(vPropNames)
    Debug.Print("    Property Name : " & vPropNames(J))
    Debug.Print("    Property Value : " & Cutlist.GetCustomPropertyValue2(vPropNames(J), nType, nLink))
    Debug.Print("    Type : " & nType)
    Debug.Print("    Link : " & nLink)
Next

Debug.Print("_____")
Debug.Print("SET NEW CUSTOM PROPERTY VALUE FOR Testing1")
Debug.Print("    Property Value Before Setting: " & Cutlist.GetCustomPropertyValue2("Testing1", nType, nLink))
Debug.Print("    New Value Set? " & Cutlist.SetCustomProperty("Testing1", "Verify3"))
Debug.Print("    Property Value After Setting : " & Cutlist.GetCustomPropertyValue2("Testing1", nType, nLink))
Debug.Print("    GET CUSTOM PROPERTIES")

vPropNames = Cutlist.GetCustomPropertyNames

For J = 0 To UBound(vPropNames)
    Debug.Print("    Property Name : " & vPropNames(J))
    Debug.Print("    Property Value : " & Cutlist.GetCustomPropertyValue2(vPropNames(J), nType, nLink))
    Debug.Print("    Type : " & nType)
    Debug.Print("    Link : " & nLink)
Next

Debug.Print("_____")
Debug.Print("DELETE CUSTOM PROPERTY Testing1")
Debug.Print("    Delete Property Value? " & Cutlist.DeleteCustomProperty("Testing1"))

vPropNames = Cutlist.GetCustomPropertyNames
If Not (IsNothing(vPropNames)) Then
    Debug.Print("    GET CUSTOM PROPERTIES")
    For J = 0 To UBound(vPropNames)
        Debug.Print("    Property Name : " & vPropNames(J))
        Debug.Print("    Property Value : " & Cutlist.GetCustomPropertyValue2(vPropNames(J), nType, nLink))
        Debug.Print("    Type : " & nType)
        Debug.Print("    Link : " & nLink)
    Next
End If

Debug.Print("_____")

'swDocument10.Save()
swDocument10.CloseDoc()

End Sub
End Module

```

Get 3D Content Central Stream Files Example (C#)

This example builds a design table and configuration rules into a SOLIDWORKS document and then demonstrates how to get the configuration rules out in two XML formats:

- * original (3DCC) format
- * generic (multi-use) format

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
//    topic and ensure that the required DLLs have been registered.
// 2. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
//    interop assembly is loaded in the project.
//    (Right-click the project in Project Explorer, click Add Reference,
//    click the interop assembly on the .NET tab, or browse for the DLL
//    in install_dir\api\redist.)
// 3. Open the specified document and add a design table
//    (Insert > Tables > Design Table).
// 4. Click the green check mark to accept the design table defaults.
// 5. Multi-select the dimensions or parameters in the document
//    to use in the design table.
// 6. Click the ConfigurationManager tab and right-click the
//    configuration name at the top.
// 7. Select Configuration Publisher on the shortcut menu.
// 8. Create rules that are functions of design table parameters,
//    setting values as appropriate.
//    (To learn how to create rules, read the 3D Content Central
//    online help: http://www.3dcontentcentral.com.)
// 9. Click Apply and Close.
// 10. Save the document to another name.
//    NOTE: Because the specified document is used in SOLIDWORKS tutorials,
//    do not save changes to the original file name.
// 11. Update the sDocFileName in the code below to reference the new
//    document.
// 12. Update sLicenseKey in the code below with your license key.
//
// Postconditions:
// 1. Inspect the Immediate window error codes.
// 2. If error codes are swDmXmlDataErrorNone for both format types,
//    inspect the directory where the part is located to find
//    motor casing_generic.xml and motor casing.xml.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

partial class SolidWorksMacro
{
    public void Main()
    {
        const string sLicenseKey = "your_license_key";
        //Specify your license key

        const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\motor casing.sldasm";

        SwDMClassFactory swClassFact = default(SwDMClassFactory);
        SwDMAApplication swDocMgr = default(SwDMAApplication);
        SwDMDocument14 swDoc = default(SwDMDocument14);
        SwDmDocumentType nDocType = default(SwDmDocumentType);
    }
}
```

```

string sPathName = null;
string gPathName = null;
SwDmDocumentOpenError nRetVal = 0;
// Determine type of SOLIDWORKS file based on file extension
if (String.Equals(sDocFileName.Substring((sDocFileName.Length - 6)), "sldprt"))
{
    nDocType = SwDmDocumentType.swDmDocumentPart;
}
else if (String.Equals(sDocFileName.Substring((sDocFileName.Length - 6)), "sldasm"))
{
    nDocType = SwDmDocumentType.swDmDocumentAssembly;
}
else if (String.Equals(sDocFileName.Substring((sDocFileName.Length - 6)), "slddrw"))
{
    nDocType = SwDmDocumentType.swDmDocumentDrawing;
}
else
{
    // Probably not a SW file
    nDocType = SwDmDocumentType.swDmDocumentUnknown;
    // So cannot open
    return;
}
// Strip off SOLIDWORKS file extension (sldxxx)
// and add XML extension (xml)
sPathName = sDocFileName.Substring(0, (sDocFileName.Length - 7));
sPathName = sPathName + ".xml";
gPathName = sDocFileName.Substring(0, (sDocFileName.Length - 7));
gPathName = gPathName + "_generic.xml";
swClassFact = new SwDMClassFactory();
swDocMgr = swClassFact.GetApplication(sLicenseKey);
swDoc = (SwDMDocument14)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);
SwDmXmlDataError Err = default(SwDmXmlDataError);
Err = swDoc.Get3DCCStreamFile(sPathName);
Debug.Print("Original (3DCC) format SwDmXmlDataError: " + Err);
Err = swDoc.Get3DCCGenericStreamFile(gPathName);
Debug.Print("Generic format SwDmXmlDataError: " + Err);
swDoc.CloseDoc();
}
public SldWorks swApp;
}

```

Get 3D Content Central Stream Files Example (VB.NET)

This example builds a design table and configuration rules into a SOLIDWORKS document and then demonstrates how to get the configuration rules out in two XML formats:

- * original (3DCC) format
- * generic (multi-use) format

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started
'    topic and ensure that the required DLLs have been registered.
' 2. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
'    interop assembly is loaded in the project.
'    (Right-click the project in Project Explorer, click Add Reference,
'    click the interop assembly on the .NET tab, or browse for
'    the DLL in install_dir\api\redist.)
' 3. Open the document specified in sDocFileName below
'    and add a design table (Insert > Tables > Design Table).
' 4. Click the green check mark to accept the design table defaults.
' 5. Multi-select the dimensions or parameters in the document
'    to use in the design table.
' 6. Click the ConfigurationManager tab and right-click the
'    configuration name at the top.
' 7. Select Configuration Publisher on the shortcut menu.
' 8. Create rules that are functions of design table parameters,
'    setting values as appropriate.
'    (To learn how to create rules, read the 3D Content Central online
'    help: http://www.3dcontentcentral.com.)
' 9. Click Apply and Close.
' 10. Save the document to another name.
'     NOTE: Because the specified document is used in SOLIDWORKS
'     tutorials, do not save changes to the original file name.
' 11. Update the sDocFileName in the code below to reference the
'     new document.
' 12. Update sLicenseKey in the code below with your license key.
'
' Postconditions:
' 1. Inspect the Immediate window error codes.
' 2. If error codes are 0, inspect the directory where
'    the part is located to find
'    motor casing_generic.xml and motor casing.xml.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Sub main()
        Const sLicenseKey As String = "your_license_key" 'Specify your license key
        Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\motor
casing.SLDASM"

        Dim swClassFact As SwDMClassFactory
        Dim swDocMgr As SwDMApplication
        Dim swDoc As SwDMDocument14
        Dim nDocType As SwDmDocumentType
        Dim sPathName As String
        Dim gPathName As String
        Dim nRetVal As Integer

        ' Determine type of SOLIDWORKS file based on file extension

```

```

If InStr(LCase(sDocFileName), "sldprt") > 0 Then
    nDocType = SwDmDocumentType.swDmDocumentPart
ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
    nDocType = SwDmDocumentType.swDmDocumentAssembly
ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
    nDocType = SwDmDocumentType.swDmDocumentDrawing
Else
    ' Probably not a SW file
    nDocType = SwDmDocumentType.swDmDocumentUnknown
    ' So cannot open
    Exit Sub
End If

' Strip off SOLIDWORKS file extension (sldxxx)
' and add XML extension (xml)
sPathName = Left(sDocFileName, Len(sDocFileName) - 7)
sPathName = sPathName + ".xml"
gPathName = Left(sDocFileName, Len(sDocFileName) - 7)
gPathName = gPathName + "_generic.xml"
swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
swDocMgr = swClassFact.GetApplication(sLicenseKey)
swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal)
Dim Err As SwDmXmlDataError
Err = swDoc.Get3DCCStreamFile(sPathName)
Debug.Print("Original (3DCC) format SwDmXmlDataError: " & Err)
Err = swDoc.Get3DCCGenericStreamFile(gPathName)
Debug.Print("Generic format SwDmXmlDataError: " & Err)
swDoc.CloseDoc()

End Sub

Public swApp As SldWorks

End Class

```

Get All External References for Drawing Example (C#)

This example shows how to get the external references for a drawing document.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API
//    Getting Started topic and ensure that
//    the required DLLs have been registered.
// 2. Copy and paste this module into a C#
//    console application in Microsoft Visual Studio.
// 3. Substitute the name of a drawing that has at least one custom property
//    for drawing_document_with_external_references.
// 4. Add the SolidWorks.Interop.swdocumentmgr.dll
//    reference to the project.
// 5. Substitute your_license_code with your SOLIDWORKS
//    Document Manager license key.
// 6. Compile and run this program in Debug mode.
//
// Postconditions: Inspect the Immediate Window.
//-----

using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Text;

class Program
{
    static void Main(string[] args)
    {
        const string sLicenseKey = "your_license_code";
        //Specify your license key
        const string sDocFileName = "drawing_document_with_external_references";

        SwDMClassFactory swClassFact = default(SwDMClassFactory);
        SwDMAApplication swDocMgr = default(SwDMAApplication);
        SwDMDocument23 swDoc = default(SwDMDocument23);
        SwDMSearchOption swSearchOpt = default(SwDMSearchOption);
        SwDmDocumentType nDocType = 0;
        SwDmDocumentOpenError nRetVal = 0;
        string[] vDependArr = null;

        SwDMDocumentLicenseType_e licenseType = 0;

        // Determine type of SOLIDWORKS file based on file extension

        if (sDocFileName.EndsWith("slddrw"))
        {
            nDocType = SwDmDocumentType.swDmDocumentDrawing;
        }
        else
        {
            // Not a drawing,
            nDocType = SwDmDocumentType.swDmDocumentUnknown;

            // So do not open
            return;
        }

        swClassFact = new SwDMClassFactory();
        swDocMgr = (SwDMAApplication)swClassFact.GetApplication(sLicenseKey);
        swSearchOpt = swDocMgr.GetSearchOptionObject();
        swDoc = (SwDMDocument23)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);
        Debug.Assert(SwDmDocumentOpenError.swDmDocumentOpenErrorNone == nRetVal);

        Debug.Print("File = " + swDoc.FullName);

        licenseType = (SwDMDocumentLicenseType_e)swDoc.GetLicenseType();

        Debug.Print("License type as defined in SwDMDocumentLicenseType_e: " + licenseType);

        string bsFileTime = "";
        string bsLWFileTime = "";
        swDoc.GetFileAvgTime(out bsFileTime, out bsLWFileTime);
        Debug.Print("File open time: " + bsFileTime);
        Debug.Print("Lightweight open time: " + bsLWFileTime);

        string[] custprops = null;
        custprops = swDoc.GetCustomPropertyNames();

        bool isEditable = false;
```

```

swDoc.IsCustomPropertyEditable(custprops[0], out isEditable);
Debug.Print("Custom property " + custprops[0] + " is editable: " + isEditable);

object vBrokenRefs = null;
object vIsVirtuals = null;
object vTimeStamps = null;
object vIsImported = null;

vDependArr = (string[])swDoc.GetAllExternalReferences5(swSearchOpt, out vBrokenRefs, out vIsVirtuals, out vTimeStamps, out
vIsImported);

if ((vDependArr == null)) return;

Debug.Print("External references");
foreach (string vDepend in vDependArr)
{
    Debug.Print(" " + vDepend);
}
Debug.Print("Reference statuses as defined in swDmReferenceStatus");
foreach (SwDmReferenceStatus brokenRef in (SwDmReferenceStatus[])vBrokenRefs)
{
    Debug.Print(" " + brokenRef);
}
Debug.Print("Virtual components?");
foreach (Boolean isVirtual in (Boolean[])vIsVirtuals)
{
    Debug.Print(" " + isVirtual);
}
Debug.Print("Timestamps");
foreach (Double timeStamp in (Int32[])vTimeStamps)
{
    Debug.Print(" " + timeStamp);
}
Debug.Print("Imported components (empty if component is not imported)");
foreach (string isImported_loopVariable in (string[])vIsImported)
{
    Debug.Print(" Imported component: " + isImported_loopVariable);
}
}
}

```

Get All External References for Drawing Example (VB.NET)

This example shows how to get the external references for a drawing document.

```
'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
'    Getting Started topic and ensure that the
'    required DLLs are registered.
' 2. Copy and paste this module into a VB.NET
'    console application in Microsoft Visual Studio.
' 3. Substitute the name of your drawing document that has at least one
'    custom property for drawing_document_with_external_references.
' 4. Add the SolidWorks.Interop.swdocumentmgr.dll
'    reference to the project.
' 5. Substitute your_license_code with your
'    SOLIDWORKS Document Manager license key.
' 6. Compile and run this program in Debug mode.
'
' Postconditions: Inspect the Immediate window.
'-----
```

Imports SolidWorks.Interop.swdocumentmgr
Module Module1

Sub main()

```
    Const sLicenseKey As String = "your_license_code" 'Specify your license key
    Const sDocFileName As String = "drawing_document_with_external_references"
```

```
    Dim swClassFact As SwDMClassFactory
    Dim swDocMgr As SwDMAApplication
    Dim swDoc As SwDMDocument23
    Dim swSearchOpt As SwDMSearchOption
    Dim nDocType As Integer
    Dim nRetVal As Integer
    Dim vDependArr As Object
    Dim vDepend As Object
    Dim vBrokenRefs As Object
    Dim vIsVirtuals As Object
    Dim vTimeStamps As Object
    Dim vIsImported As Object
    Dim licenseType As Integer
```

```
    ' Determine if document is a drawing
    If InStr(LCase(sDocFileName), "slddrw") > 0 Then
        nDocType = SwDMDocumentType.swDmDocumentDrawing
    Else
        ' Not a drawings file,
        nDocType = SwDMDocumentType.swDmDocumentUnknown
        ' so do not open
        Exit Sub
    End If
```

```
    swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
    swDocMgr = swClassFact.GetApplication(sLicenseKey)
    swSearchOpt = swDocMgr.GetSearchOptionObject
    swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal) :
    Debug.Assert(SwDMDocumentOpenError.swDmDocumentOpenErrorNone = nRetVal)
```

```
    Debug.Print("File = " & swDoc.FullName)
```

```
    licenseType = swDoc.GetLicenseType
    Debug.Print("License type as defined in swDMDocumentLicenseType_e: " & licenseType)
```

```
    Dim bsFileTime As String = ""
    Dim bsLWFileTime As String = ""
    swDoc.GetFileAvgTime(bsFileTime, bsLWFileTime)
    Debug.Print("File open time: " & bsFileTime)
    Debug.Print("Lightweight open time: " & bsLWFileTime)
```

```
    Dim custprops As Object
    custprops = swDoc.GetCustomPropertyNames()
```

```
    Dim isEditable As Boolean
    swDoc.IsCustomPropertyEditable(custprops(0), isEditable)
    Debug.Print("Custom property " & custprops(0) & " is editable: " & isEditable)
```

```
    vBrokenRefs = Nothing
    Dim brokenRef As SwDMReferenceStatus
    vIsVirtuals = Nothing
    Dim isVirtual As Boolean
    vTimeStamps = Nothing
    Dim timeStamp As Double
    vIsImported = Nothing
```

```

    Dim isImported As String

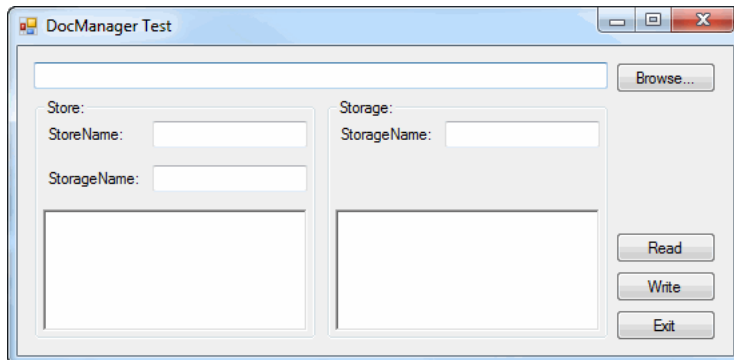
    vDependArr = swDoc.GetAllExternalReferences5(swSearchOpt, vBrokenRefs, vIsVirtuals, vTimeStamps, vIsImported) : If
    IsNothing(vDependArr) Then Exit Sub

    Debug.Print("External references")
    For Each vDepend In vDependArr
        Debug.Print(" " & vDepend)
    Next vDepend
    Debug.Print("Reference statuses as defined in swDmReferenceStatus")
    For Each brokenRef In vBrokenRefs
        Debug.Print(" " & brokenRef)
    Next brokenRef
    Debug.Print("Virtual components?")
    For Each isVirtual In vIsVirtuals
        Debug.Print(" " & isVirtual)
    Next isVirtual
    Debug.Print("Timestamps")
    For Each timeStamp In vTimeStamps
        Debug.Print(" " & timeStamp)
    Next timeStamp
    Debug.Print("Imported components (empty if component is not imported)")
    For Each isImported In vIsImported
        Debug.Print(" Imported component: " & isImported)
    Next isImported
End Sub
End Module

```

Read and Write to Third-party Storage Example (C#)

This example shows how to read and write to third-party storage in a document.



```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
//    topic and ensure that the required DLLs are registered.
// 2. Start Microsoft Visual Studio 2010.
// 3. Click File > New > Project > Visual C# > Windows Forms Application.
//    a. Type SwDocMgrStorageApp in Name.
//    b. Click Browse and navigate to the folder where to create
//       the project.
//    c. Click OK.
// 4. Add references to the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click:
//       install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
//    e. Click Open.
//    f. Click Add.
//    g. Click Browse.
//    h. Click:
//       C:\Program Files\Microsoft Visual Studio 11.0\Common7\IDE\Remote Debugger\x64\Microsoft.VisualStudio.OLE.Interop.dll
//    i. Click Open.
//    j. Click Add.
//    k. Click Close.
// 5. Click Show All Files in the Solution Explorer toolbar and expand
//    Form1.cs in the Solution Explorer.
// 6. Replace the code in Form1.cs with this code.
// 7. Replace the code in Form1.Designer.cs with this code.
// 8. Substitute your_license_key with your SOLIDWORKS Document
//    Manager license key.
//
// Postconditions:
// 1. Displays a dialog.
// 2. Click Browse.
//    a. Locate and click a SOLIDWORKS document.
//    b. Click Open.
// 3. Type temp in StoreName and in both StorageName fields.
// 4. Type Some third-party text in both text boxes.
// 5. Click Write.
// 6. Click Exit.
// 7. Press F5.
// 8. In the dialog, click Browse.
//    a. Locate and click the document you clicked in step 2a.
//    b. Click Open.
// 9. Type temp in StoreName and in both StorageName fields.
// 10. Click Read.
// 11. Displays Some third-party text in both text boxes.
// 12. Click Exit.
//-----
//Form1.cs
```

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using SwDocumentMgr;
using Microsoft.VisualStudio.OLE.Interop;
using System.IO;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace SwDocMgrStorageApp
```

```

{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        SwDmDocument19 swDoc19;
        SwDmApplication swDocMgr;

        SwDmClassFactory swClassFact;

        string sLicenseKey = "your_license_key";
        private void btnFile_Click(System.Object sender, System.EventArgs e)
        {
            OpenFileDialog1.Filter = "SOLIDWORKS Files (*.sldprt;*.sldasm;*.slddrw)|*.sldprt;*.sldasm;*.slddrw|SOLIDWORKS Part (*.sldprt)|*.sldprt|S
            if (OpenFileDialog1.ShowDialog() == System.Windows.Forms.DialogResult.OK) {
                txtFileName.Text = OpenFileDialog1.FileName;
            }
        }

        private void btnOpen_Click(System.Object sender, System.EventArgs e)
        {
            OpenFile();
            if ((swDoc19 != null)) {
                if (txtStorageName.Text != string.Empty)
                    Read3rdPartyStorageData();
                if (txtStoreName.Text != string.Empty && txtStreamName.Text != string.Empty)
                    Read3rdPartyStorageStoreData();
            }
            CloseFile();
            this.Refresh();
        }

        private void btnSave_Click(System.Object sender, System.EventArgs e)
        {
            OpenFile();
            if ((swDoc19 != null)) {
                if (txtStorageName.Text != string.Empty)
                    Write3rdPartyStorageData();
                if (txtStoreName.Text != string.Empty && txtStreamName.Text != string.Empty)
                    Write3rdPartyStorageStoreData();
            }
            rtbStorage.Text = string.Empty;
            rtbStore.Text = string.Empty;
            CloseFile();
            this.Refresh();
        }

        private void btnExit_Click(System.Object sender, System.EventArgs e)
        {
            this.Close();
        }
        private void OpenFile()
        {
            if ((swDoc19 != null)) {
                swDoc19.CloseDoc();
                swDoc19 = null;
            }

            string sDocFileName = null;
            SwDmDocumentType nDocType = 0;
            SwDmDocumentOpenError nRetVal = 0;

            sDocFileName = txtFileName.Text;
            if (sDocFileName.ToLower().EndsWith(".sldprt"))
            {
                nDocType = SwDmDocumentType.swDmDocumentPart;
            }
            else if (sDocFileName.ToLower().EndsWith(".sldasm"))
            {
                nDocType = SwDmDocumentType.swDmDocumentAssembly;
            }
            else if (sDocFileName.ToLower().EndsWith(".slddrw"))
            {
                nDocType = SwDmDocumentType.swDmDocumentDrawing;
            }
            else
            {
                // Not a SOLIDWORKS file
                nDocType = SwDmDocumentType.swDmDocumentUnknown;
            }

            return;
        }

        swClassFact = new SwDmClassFactory();
        swDocMgr = swClassFact.GetApplication(sLicenseKey) as SwDmApplication ;
        swDoc19 = swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal) as SwDmDocument19;
        Debug.Assert(SwDmDocumentOpenError.swDmDocumentOpenErrorNone == nRetVal);
    }
}

```

```

private void CloseFile()
{
    if ((swDoc19 != null))
    {
        swDoc19.CloseDoc();
        System.Runtime.InteropServices.Marshal.ReleaseComObject(swDoc19);
        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();
        GC.WaitForPendingFinalizers();
        swDoc19 = null;
    }
}

private void Read3rdPartyStorageData()
{
    try
    {
        IStream pIStream = default(IStream);
        pIStream = swDoc19.Get3rdPartyStorage(txtStorageName.Text, false);
        if (pIStream == null)
        {
            rtbStorage.Text = "Error: " + "Storage " + txtStorageName.Text + " not found";
            return;
        }
        string strStream = null;
        byte[] byteArray = new byte[1000];
        uint outlength = 0;
        pIStream.Read(byteArray, (uint)byteArray.Length, out outlength);
        strStream = Encoding.UTF8.GetString(byteArray);
        rtbStorage.Text = strStream.Substring(0, (int)outlength);
        swDoc19.Release3rdPartyStorage(txtStorageName.Text);
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream);
        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();
        GC.WaitForPendingFinalizers();
    }
    catch (Exception ex)
    {
        rtbStorage.Text = "Error: " + ex.Message;
    }
}

private void Read3rdPartyStorageStoreData()
{
    try
    {
        IStorage storage = default(IStorage);
        storage = swDoc19.Get3rdPartyStorageStore(txtStoreName.Text, false);
        if (storage == null)
        {
            rtbStore.Text = "Error: " + "Store " + txtStoreName.Text + " not found";
            return;
        }
        IStream pIStream = default(IStream);
        int grfmode = 0;
        grfmode |= (int)STGM.SHARE_EXCLUSIVE;
        storage.OpenStream(txtStreamName.Text, IntPtr.Zero, (uint)grfmode, 0, out pIStream);
        if (pIStream == null)
        {
            rtbStore.Text = "Error: " + "Stream " + txtStreamName.Text + " not found";
            return;
        }
        string strStream = null;
        byte[] byteArray = new byte[1000];
        uint outlength = 0;
        pIStream.Read(byteArray, (uint)byteArray.Length, out outlength);
        strStream = Encoding.UTF8.GetString(byteArray);
        rtbStore.Text = strStream;
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream);
        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();
        GC.WaitForPendingFinalizers();
        swDoc19.Release3rdPartyStorageStore(txtStoreName.Text);
    }
    catch (Exception ex)
    {
        rtbStore.Text = "Error: " + ex.Message;
    }
}

private void Write3rdPartyStorageData()
{
    try
    {
        IStream pIStream = default(IStream);
        pIStream = swDoc19.Get3rdPartyStorage(txtStorageName.Text, true);
        if (pIStream == null)
        {
            rtbStorage.Text = "Error: " + "Storage " + txtStorageName.Text + " not found";
            return;
        }
        byte[] strStream = null;
        strStream = Encoding.UTF8.GetBytes(rtbStorage.Text);
        uint outlength = 0;
        pIStream.Write(strStream, (uint)strStream.Length, out outlength);
    }
}

```

```

        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream);
        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();
        GC.WaitForPendingFinalizers();
        swDoc19.Release3rdPartyStorage(txtStorageName.Text);
        swDoc19.Save();
    }
    catch(Exception ex)
    {
        rtbStorage.Text = "Error: " + ex.Message;
    }
}

private void Write3rdPartyStorageStoreData()
{
    try
    {
        IStorage storage = default(IStorage);
        storage = swDoc19.Get3rdPartyStorageStore(txtStoreName.Text, true);
        if (storage == null)
        {
            rtbStore.Text = "Error: " + "Store " + txtStoreName.Text + " not found";
            return;
        }
        IStream pIStream = default(IStream);
        int grfmode = 0;
        grfmode |= (int)STGM.SHARE_EXCLUSIVE;
        grfmode |= (int)STGM.READWRITE;
        grfmode |= (int)STGM.CREATE;
        storage.CreateStream(txtStreamName.Text, (uint)grfmode, 0, 0, out pIStream);
        if (pIStream == null)
        {
            rtbStore.Text = "Error: " + "Stream " + txtStreamName.Text + " not found";
            return;
        }
        byte[] strStream = null;
        strStream = Encoding.UTF8.GetBytes(rtbStore.Text);
        uint outlength = 0;
        pIStream.Write(strStream, (uint)strStream.Length, out outlength);
        System.Runtime.InteropServices.Marshal.ReleaseComObject(storage);
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream);
        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();
        GC.WaitForPendingFinalizers();
        swDoc19.Release3rdPartyStorageStore(txtStoreName.Text);
        swDoc19.Save();
    }
    catch(Exception ex)
    {
        rtbStore.Text = "Error: " + ex.Message;
    }
}

[Flags]
public enum STGM : uint
{
    DIRECT = 0x00000000,
    TRANSACTED = 0x00010000,
    SIMPLE = 0x00000000,
    READ = 0x00000000,
    WRITE = 0x00000001,
    READWRITE = 0x00000002,
    SHARE_DENY_NONE = 0x00000040,
    SHARE_DENY_READ = 0x00000030,
    SHARE_DENY_WRITE = 0x00000020,
    SHARE_EXCLUSIVE = 0x00000010,
    PRIORITY = 0x00040000,
    DELETEONRELEASE = 0x04000000,
    NOSCRATCH = 0x00100000,
    CREATE = 0x00010000,
    CONVERT = 0x00020000,
    FAILIF THERE = 0x00000000,
    NOSNAPSHOT = 0x00200000,
    DIRECT_SWMR = 0x00400000,
}
}

//Form1.Designer.cs
namespace SwDocMgrStorageApp
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
    }
}

```

```

{
    if (disposing && (components != null))
    {
        components.Dispose();
    }
    base.Dispose(disposing);
}

#region Windows Form Designer generated code

/// <summary>
/// Required method for Designer support - do not modify
/// the contents of this method with the code editor.
/// </summary>
private void InitializeComponent()
{
    this.rtbStorage = new System.Windows.Forms.RichTextBox();
    this.rtbStore = new System.Windows.Forms.RichTextBox();
    this.btnExit = new System.Windows.Forms.Button();
    this.btnSave = new System.Windows.Forms.Button();
    this.btnOpen = new System.Windows.Forms.Button();
    this.btnFile = new System.Windows.Forms.Button();
    this.OpenFileDialog1 = new System.Windows.Forms.OpenFileDialog();
    this.txtFileName = new System.Windows.Forms.TextBox();
    this.groupBox1 = new System.Windows.Forms.GroupBox();
    this.txtStoreName = new System.Windows.Forms.TextBox();
    this.label1 = new System.Windows.Forms.Label();
    this.label3 = new System.Windows.Forms.Label();
    this.txtStreamName = new System.Windows.Forms.TextBox();
    this.groupBox2 = new System.Windows.Forms.GroupBox();
    this.label5 = new System.Windows.Forms.Label();
    this.txtStorageName = new System.Windows.Forms.TextBox();
    this.groupBox1.SuspendLayout();
    this.groupBox2.SuspendLayout();
    this.SuspendLayout();
    //
    // rtbStorage
    //
    this.rtbStorage.Location = new System.Drawing.Point(6, 83);
    this.rtbStorage.Name = "rtbStorage";
    this.rtbStorage.Size = new System.Drawing.Size(198, 91);
    this.rtbStorage.TabIndex = 15;
    this.rtbStorage.Text = "";
    //
    // rtbStore
    //
    this.rtbStore.Location = new System.Drawing.Point(6, 83);
    this.rtbStore.Name = "rtbStore";
    this.rtbStore.Size = new System.Drawing.Size(198, 91);
    this.rtbStore.TabIndex = 14;
    this.rtbStore.Text = "";
    //
    // btnExit
    //
    this.btnExit.Location = new System.Drawing.Point(448, 197);
    this.btnExit.Name = "btnExit";
    this.btnExit.Size = new System.Drawing.Size(75, 23);
    this.btnExit.TabIndex = 13;
    this.btnExit.Text = "Exit";
    this.btnExit.UseVisualStyleBackColor = true;
    this.btnExit.Click += new System.EventHandler(this.btnExit_Click);
    //
    // btnSave
    //
    this.btnSave.Location = new System.Drawing.Point(448, 168);
    this.btnSave.Name = "btnSave";
    this.btnSave.Size = new System.Drawing.Size(75, 23);
    this.btnSave.TabIndex = 12;
    this.btnSave.Text = "Write";
    this.btnSave.UseVisualStyleBackColor = true;
    this.btnSave.Click += new System.EventHandler(this.btnSave_Click);
    //
    // btnOpen
    //
    this.btnOpen.Location = new System.Drawing.Point(448, 139);
    this.btnOpen.Name = "btnOpen";
    this.btnOpen.Size = new System.Drawing.Size(75, 23);
    this.btnOpen.TabIndex = 11;
    this.btnOpen.Text = "Read";
    this.btnOpen.UseVisualStyleBackColor = true;
    this.btnOpen.Click += new System.EventHandler(this.btnOpen_Click);
    //
    // btnFile
    //
    this.btnFile.Location = new System.Drawing.Point(448, 12);
    this.btnFile.Name = "btnFile";
    this.btnFile.Size = new System.Drawing.Size(75, 23);
    this.btnFile.TabIndex = 10;
    this.btnFile.Text = "Browse...";
    this.btnFile.UseVisualStyleBackColor = true;
    this.btnFile.Click += new System.EventHandler(this.btnFile_Click);
    //
    // OpenFileDialog1
    //
    this.OpenFileDialog1.FileName = "OpenFileDialog1";
    //

```

```

// txtFileName
//
this.txtFileName.Location = new System.Drawing.Point(12, 12);
this.txtFileName.Name = "txtFileName";
this.txtFileName.Size = new System.Drawing.Size(430, 20);
this.txtFileName.TabIndex = 9;
//
// groupBox1
//
this.groupBox1.Controls.Add(this.label3);
this.groupBox1.Controls.Add(this.txtStreamName);
this.groupBox1.Controls.Add(this.label1);
this.groupBox1.Controls.Add(this.txtStoreName);
this.groupBox1.Controls.Add(this.rtbStore);
this.groupBox1.Location = new System.Drawing.Point(13, 39);
this.groupBox1.Name = "groupBox1";
this.groupBox1.Size = new System.Drawing.Size(210, 180);
this.groupBox1.TabIndex = 18;
this.groupBox1.TabStop = false;
this.groupBox1.Text = "Store:";
//
// txtStoreName
//
this.txtStoreName.Location = new System.Drawing.Point(88, 17);
this.txtStoreName.Name = "txtStoreName";
this.txtStoreName.Size = new System.Drawing.Size(116, 20);
this.txtStoreName.TabIndex = 15;
//
// label1
//
this.label1.AutoSize = true;
this.label1.Location = new System.Drawing.Point(7, 20);
this.label1.Name = "label1";
this.label1.Size = new System.Drawing.Size(63, 13);
this.label1.TabIndex = 16;
this.label1.Text = "StoreName:";
//
// label3
//
this.label3.AutoSize = true;
this.label3.Location = new System.Drawing.Point(7, 53);
this.label3.Name = "label3";
this.label3.Size = new System.Drawing.Size(75, 13);
this.label3.TabIndex = 18;
this.label3.Text = "StorageName:";
//
// txtStreamName
//
this.txtStreamName.Location = new System.Drawing.Point(88, 50);
this.txtStreamName.Name = "txtStreamName";
this.txtStreamName.Size = new System.Drawing.Size(116, 20);
this.txtStreamName.TabIndex = 17;
//
// groupBox2
//
this.groupBox2.Controls.Add(this.rtbStorage);
this.groupBox2.Controls.Add(this.label5);
this.groupBox2.Controls.Add(this.txtStorageName);
this.groupBox2.Location = new System.Drawing.Point(232, 39);
this.groupBox2.Name = "groupBox2";
this.groupBox2.Size = new System.Drawing.Size(210, 180);
this.groupBox2.TabIndex = 19;
this.groupBox2.TabStop = false;
this.groupBox2.Text = "Storage:";
//
// label5
//
this.label5.AutoSize = true;
this.label5.Location = new System.Drawing.Point(7, 20);
this.label5.Name = "label5";
this.label5.Size = new System.Drawing.Size(75, 13);
this.label5.TabIndex = 16;
this.label5.Text = "StorageName:";
//
// txtStorageName
//
this.txtStorageName.Location = new System.Drawing.Point(88, 17);
this.txtStorageName.Name = "txtStorageName";
this.txtStorageName.Size = new System.Drawing.Size(116, 20);
this.txtStorageName.TabIndex = 15;
//
// Form1
//
this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
this.AutoScaleModeMode = System.Windows.Forms.AutoScaleModeMode.Font;
this.ClientSize = new System.Drawing.Size(536, 231);
this.Controls.Add(this.btnExit);
this.Controls.Add(this.btnSave);
this.Controls.Add(this.btnOpen);
this.Controls.Add(this.btnFile);
this.Controls.Add(this.txtFileName);
this.Controls.Add(this.groupBox1);
this.Controls.Add(this.groupBox2);
this.MaximumSize = new System.Drawing.Size(552, 269);
this.MinimumSize = new System.Drawing.Size(552, 269);
this.Name = "Form1";

```

```
this.Text = "DocManager Test";
this.groupBox1.ResumeLayout(false);
this.groupBox1.PerformLayout();
this.groupBox2.ResumeLayout(false);
this.groupBox2.PerformLayout();
this.ResumeLayout(false);
this.PerformLayout();
```

```
}
```

```
#endregion
```

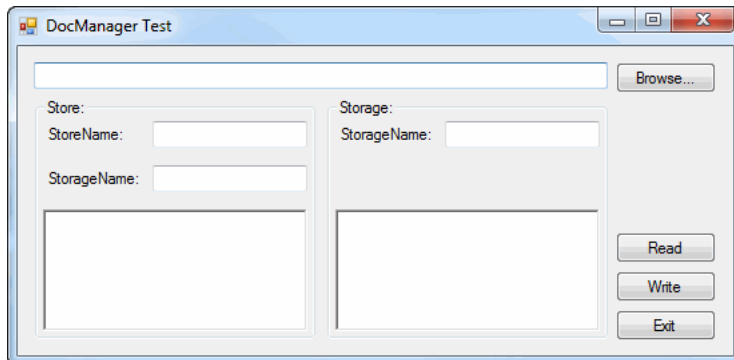
```
internal System.Windows.Forms.RichTextBox rtbStorage;
internal System.Windows.Forms.RichTextBox rtbStore;
internal System.Windows.Forms.Button btnExit;
internal System.Windows.Forms.Button btnSave;
internal System.Windows.Forms.Button btnOpen;
internal System.Windows.Forms.Button btnFile;
internal System.Windows.Forms.OpenFileDialog openFileDialog1;
internal System.Windows.Forms.TextBox txtFileName;
private System.Windows.Forms.GroupBox groupBox1;
private System.Windows.Forms.Label label3;
private System.Windows.Forms.TextBox txtStreamName;
private System.Windows.Forms.Label label1;
private System.Windows.Forms.TextBox txtStoreName;
private System.Windows.Forms.GroupBox groupBox2;
private System.Windows.Forms.Label label5;
private System.Windows.Forms.TextBox txtStorageName;
```

```
}
```

```
}
```

Read and Write to Third-party Storage Example (VB.NET)

This example shows how to read and write to third-party storage in a document.



Preconditions:

1. Read the SOLIDWORKS Document Manager API Getting Started topic and ensure that the required DLLs are registered.
2. Start Microsoft Visual Studio 2010.
3. Click **File > New > Project > Visual Basic > Windows Forms Application**.
 - a. Type **SwDocMgrStorageApp_VBNET** in **Name**.
 - b. Click **Browse** and navigate to the folder where to create the project.
 - c. Click **OK**.
4. Add references to the project:
 - a. Right-click the solution in Solution Explorer.
 - b. Click **Add Reference**.
 - c. Click **Browse**.
 - d. Click:


```
install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
```
 - e. Click **Open**.
 - f. Click **Add**.
 - g. Click **Browse**.
 - h. Click:


```
C:\Program Files\Microsoft Visual Studio 11.0\Common7\IDE\Remote Debugger\x64\Microsoft.VisualStudio.OLE.Interop.dll
```
 - i. Click **Open**.
 - j. Click **Add**.
 - k. Click **Close**.
5. Click **Show All Files** in the Solution Explorer toolbar and expand **Form1.vb** in the Solution Explorer.
6. Replace the code in **Form1.vb** with [this code](#).
7. Replace the code in **Form1.Designer.vb** with [this code](#).
8. Substitute *your_license_key* with your SOLIDWORKS Document Manager license key.

Postconditions:

1. Displays a dialog.
2. Click **Browse**.
 - a. Locate and click a SOLIDWORKS document.
 - b. Click **Open**.
3. Type **temp** in **StoreName** and in both **StorageName** fields.
4. Type **Some third-party text** in both text boxes.
5. Click **Write**.
6. Click **Exit**.
7. Press F5.
8. In the dialog, click **Browse**.
 - a. Locate and click the document you clicked in step 2a.
 - b. Click **Open**.
9. Type **temp** in **StoreName** and in both **StorageName** fields.
10. Click **Read**.
11. Displays **Some third-party text** in both text boxes.
12. Click **Exit**.

'Form1.vb

```
Imports System.Collections.Generic
Imports System.ComponentModel
Imports System.Data
Imports System.Drawing
Imports System.Linq
Imports System.Text
Imports System.Windows.Forms
Imports SwDocumentMgr
Imports Microsoft.VisualStudio.OLE.Interop
Imports System.IO
Imports System.Diagnostics
Imports System.Runtime.InteropServices
```

```

Public Class Form1
    Inherits Form
    Public Sub New()
        InitializeComponent()
    End Sub
    Public swDoc19 As SwDMDocument19
    Public swDocMgr As SwDMApplication
    Public swClassFact As SwDMClassFactory
    Public sLicenseKey As String = "your_license_key"

    Public Sub btnFile_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnFile.Click
        OpenFileDialog1.Filter = "SOLIDWORKS Files (*.sldprt;*.sldasm;*.slddrw)|*.sldprt;*.sldasm;*.slddrw|SOLIDWORKS Part (*.sldprt)|*.*
        If OpenFileDialog1.ShowDialog() = System.Windows.Forms.DialogResult.OK Then
            txtFileName.Text = OpenFileDialog1.FileName
        End If
    End Sub

    Public Sub btnOpen_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnOpen.Click
        OpenFile()
        If (swDoc19 IsNot Nothing) Then
            If txtStorageName.Text <> String.Empty Then
                Read3rdPartyStorageData()
            End If
            If txtStoreName.Text <> String.Empty AndAlso txtStreamName.Text <> String.Empty Then
                Read3rdPartyStorageStoreData()
            End If
        End If
        CloseFile()
        Me.Refresh()
    End Sub

    Public Sub btnSave_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnSave.Click
        OpenFile()
        If (swDoc19 IsNot Nothing) Then
            If txtStorageName.Text <> String.Empty Then
                Write3rdPartyStorageData()
            End If
            If txtStoreName.Text <> String.Empty AndAlso txtStreamName.Text <> String.Empty Then
                Write3rdPartyStorageStoreData()
            End If
        End If
        rtbStorage.Text = String.Empty
        rtbStore.Text = String.Empty
        CloseFile()
        Me.Refresh()
    End Sub

    Public Sub btnExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnExit.Click
        Me.Close()
    End Sub

    Public Sub OpenFile()
        If (swDoc19 IsNot Nothing) Then
            swDoc19.CloseDoc()
            swDoc19 = Nothing
        End If

        Dim sDocFileName As String = Nothing
        Dim nDocType As SwDmDocumentType = 0
        Dim nRetVal As SwDmDocumentOpenError = 0

        sDocFileName = txtFileName.Text
        If sDocFileName.ToLower().EndsWith(".sldprt") Then
            nDocType = SwDmDocumentType.swDmDocumentPart
        ElseIf sDocFileName.ToLower().EndsWith(".sldasm") Then
            nDocType = SwDmDocumentType.swDmDocumentAssembly
        ElseIf sDocFileName.ToLower().EndsWith(".slddrw") Then
            nDocType = SwDmDocumentType.swDmDocumentDrawing
        Else
            ' Not a SOLIDWORKS file
            nDocType = SwDmDocumentType.swDmDocumentUnknown

            Return
        End If

        swClassFact = New SwDMClassFactory()
        swDocMgr = TryCast(swClassFact.GetApplication(sLicenseKey), SwDMApplication)
        swDoc19 = TryCast(swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal), SwDMDocument19)
        Debug.Assert(SwDmDocumentOpenError.swDmDocumentOpenErrorNone = nRetVal)
    End Sub

    Public Sub CloseFile()
        If (swDoc19 IsNot Nothing) Then
            swDoc19.CloseDoc()
            System.Runtime.InteropServices.Marshal.ReleaseComObject(swDoc19)
            GC.Collect()
            GC.WaitForPendingFinalizers()
            GC.Collect()
            GC.WaitForPendingFinalizers()
            swDoc19 = Nothing
        End If
    End Sub
End Class

```

```

Public Sub Read3rdPartyStorageData()
    Try
        Dim pIStream As IStream = Nothing
        pIStream = swDoc19.Get3rdPartyStorage(txtStorageName.Text, False)
        If pIStream Is Nothing Then
            rtbStorage.Text = "Error: " + "Storage " + txtStorageName.Text + " not found"
            Return
        End If
        Dim strStream As String = Nothing
        Dim byteArray As Byte() = New Byte(999) {}
        Dim outlength As UInteger = 0
        pIStream.Read(byteArray, CUInt(byteArray.Length), outlength)
        strStream = Encoding.UTF8.GetString(byteArray)
        rtbStorage.Text = strStream.Substring(0, CInt(outlength))
        swDoc19.Release3rdPartyStorage(txtStorageName.Text)
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream)
        GC.Collect()
        GC.WaitForPendingFinalizers()
        GC.Collect()
        GC.WaitForPendingFinalizers()
    Catch ex As Exception
        rtbStorage.Text = "Error: " + ex.Message
    End Try
End Sub

Public Sub Read3rdPartyStorageStoreData()
    Try
        Dim storage As IStorage = Nothing
        storage = swDoc19.Get3rdPartyStorageStore(txtStoreName.Text, False)
        If storage Is Nothing Then
            rtbStore.Text = "Error: " + "Store " + txtStoreName.Text + " not found"
            Return
        End If
        Dim pIStream As IStream = Nothing
        Dim grfmode As Integer = 0
        grfmode = grfmode Or CInt(STGM.SHARE_EXCLUSIVE)
        storage.OpenStream(txtStreamName.Text, IntPtr.Zero, CUInt(grfmode), 0, pIStream)
        If pIStream Is Nothing Then
            rtbStore.Text = "Error: " + "Stream " + txtStreamName.Text + " not found"
            Return
        End If
        Dim strStream As String = Nothing
        Dim byteArray As Byte() = New Byte(999) {}
        Dim outlength As UInteger = 0
        pIStream.Read(byteArray, CUInt(byteArray.Length), outlength)
        strStream = Encoding.UTF8.GetString(byteArray)
        rtbStore.Text = strStream
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream)
        GC.Collect()
        GC.WaitForPendingFinalizers()
        GC.Collect()
        GC.WaitForPendingFinalizers()
        swDoc19.Release3rdPartyStorageStore(txtStoreName.Text)
    Catch ex As Exception
        rtbStore.Text = "Error: " + ex.Message
    End Try
End Sub

Public Sub Write3rdPartyStorageData()
    Try
        Dim pIStream As IStream = Nothing
        pIStream = swDoc19.Get3rdPartyStorage(txtStorageName.Text, True)
        If pIStream Is Nothing Then
            rtbStorage.Text = "Error: " + "Storage " + txtStorageName.Text + " not found"
            Return
        End If
        Dim strStream As Byte() = Nothing
        strStream = Encoding.UTF8.GetBytes(rtbStorage.Text)
        Dim outlength As UInteger = 0
        pIStream.Write(strStream, CUInt(strStream.Length), outlength)
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream)
        GC.Collect()
        GC.WaitForPendingFinalizers()
        GC.Collect()
        GC.WaitForPendingFinalizers()
        swDoc19.Release3rdPartyStorage(txtStorageName.Text)
        swDoc19.Save()
    Catch ex As Exception
        rtbStorage.Text = "Error: " + ex.Message
    End Try
End Sub

Public Sub Write3rdPartyStorageStoreData()
    Try
        Dim storage As IStorage = Nothing
        storage = swDoc19.Get3rdPartyStorageStore(txtStoreName.Text, True)
        If storage Is Nothing Then
            rtbStore.Text = "Error: " + "Store " + txtStoreName.Text + " not found"
            Return
        End If
        Dim pIStream As IStream = Nothing
        Dim grfmode As Integer = 0
        grfmode = grfmode Or CInt(STGM.SHARE_EXCLUSIVE)
        grfmode = grfmode Or CInt(STGM.READWRITE)
        grfmode = grfmode Or CInt(STGM.CREATE)
        storage.CreateStream(txtStreamName.Text, CUInt(grfmode), 0, 0, pIStream)
    End Try
End Sub

```

```

        If pIStream Is Nothing Then
            rtbStore.Text = "Error: " + "Stream " + txtStreamName.Text + " not found"
            Return
        End If
        Dim strStream As Byte() = Nothing
        strStream = Encoding.UTF8.GetBytes(rtbStore.Text)
        Dim outlength As UInteger = 0
        pIStream.Write(strStream, CUInt(strStream.Length), outlength)
        System.Runtime.InteropServices.Marshal.ReleaseComObject(storage)
        System.Runtime.InteropServices.Marshal.ReleaseComObject(pIStream)
        GC.Collect()
        GC.WaitForPendingFinalizers()
        GC.Collect()
        GC.WaitForPendingFinalizers()
        swDoc19.Release3rdPartyStorageStore(txtStoreName.Text)
        swDoc19.Save()
    Catch ex As Exception
        rtbStore.Text = "Error: " + ex.Message
    End Try
End Sub

<Flags()> _
Public Enum STGM As UInteger
    DIRECT = &H0
    TRANSACTED = &H10000
    SIMPLE = &H80000000
    READ = &H0
    WRITE = &H1
    READWRITE = &H2
    SHARE_DENY_NONE = &H40
    SHARE_DENY_READ = &H30
    SHARE_DENY_WRITE = &H20
    SHARE_EXCLUSIVE = &H10
    PRIORITY = &H40000
    DELETEONRELEASE = &H40000000
    NOSCRATCH = &H1000000
    CREATE = &H1000
    CONVERT = &H20000
    FAILIFTHRE = &H0
    NOSNAPSHOT = &H2000000
    DIRECT_SWMR = &H4000000
End Enum

End Class

'Form1.Designer.vb
<Global.Microsoft.VisualBasic.CompilerServices.DesignerGenerated()> _
Partial Class Form1
    Inherits System.Windows.Forms.Form

    'Form overrides dispose to clean up the component list.
    <System.Diagnostics.DebuggerNonUserCode()> _
    Protected Overrides Sub Dispose(ByVal disposing As Boolean)
        Try
            If disposing AndAlso components IsNot Nothing Then
                components.Dispose()
            End If
        Finally
            MyBase.Dispose(disposing)
        End Try
    End Sub

    'Required by the Windows Form Designer
    Private components As System.ComponentModel.IContainer

    'NOTE: The following procedure is required by the Windows Form Designer
    'It can be modified using the Windows Form Designer.
    'Do not modify it using the code editor.
    <System.Diagnostics.DebuggerStepThrough()> _
    Private Sub InitializeComponent()

        Me.rtbStorage = New System.Windows.Forms.RichTextBox()
        Me.rtbStore = New System.Windows.Forms.RichTextBox()
        Me.btnExit = New System.Windows.Forms.Button()
        Me.btnSave = New System.Windows.Forms.Button()
        Me.btnOpen = New System.Windows.Forms.Button()
        Me.btnFile = New System.Windows.Forms.Button()
        Me.OpenFileDialog1 = New System.Windows.Forms.OpenFileDialog()
        Me.txtFileName = New System.Windows.Forms.TextBox()
        Me.groupBox1 = New System.Windows.Forms.GroupBox()
        Me.txtStoreName = New System.Windows.Forms.TextBox()
        Me.label1 = New System.Windows.Forms.Label()
        Me.label3 = New System.Windows.Forms.Label()
        Me.txtStreamName = New System.Windows.Forms.TextBox()
        Me.groupBox2 = New System.Windows.Forms.GroupBox()
        Me.label5 = New System.Windows.Forms.Label()
        Me.txtStorageName = New System.Windows.Forms.TextBox()
        Me.groupBox1.SuspendLayout()
        Me.groupBox2.SuspendLayout()
        Me.SuspendLayout()
        '
        ' rtbStorage
        '
        Me.rtbStorage.Location = New System.Drawing.Point(6, 83)
        Me.rtbStorage.Name = "rtbStorage"

```

```

Me.rtbStorage.Size = New System.Drawing.Size(198, 91)
Me.rtbStorage.TabIndex = 15
Me.rtbStorage.Text = ""
'
' rtbStore
'
Me.rtbStore.Location = New System.Drawing.Point(6, 83)
Me.rtbStore.Name = "rtbStore"
Me.rtbStore.Size = New System.Drawing.Size(198, 91)
Me.rtbStore.TabIndex = 14
Me.rtbStore.Text = ""
'
' btnExit
'
Me.btnExit.Location = New System.Drawing.Point(448, 197)
Me.btnExit.Name = "btnExit"
Me.btnExit.Size = New System.Drawing.Size(75, 23)
Me.btnExit.TabIndex = 13
Me.btnExit.Text = "Exit"
Me.btnExit.UseVisualStyleBackColor = True
'Me.btnExit.Click += New System.EventHandler(Me.btnExit_Click)
'
' btnSave
'
Me.btnSave.Location = New System.Drawing.Point(448, 168)
Me.btnSave.Name = "btnSave"
Me.btnSave.Size = New System.Drawing.Size(75, 23)
Me.btnSave.TabIndex = 12
Me.btnSave.Text = "Write"
Me.btnSave.UseVisualStyleBackColor = True
'Me.btnSave.Click += New System.EventHandler(Me.btnSave_Click)
'
' btnOpen
'
Me.btnOpen.Location = New System.Drawing.Point(448, 139)
Me.btnOpen.Name = "btnOpen"
Me.btnOpen.Size = New System.Drawing.Size(75, 23)
Me.btnOpen.TabIndex = 11
Me.btnOpen.Text = "Read"
Me.btnOpen.UseVisualStyleBackColor = True
'Me.btnOpen.Click += New System.EventHandler(Me.btnOpen_Click)
'
' btnFile
'
Me.btnFile.Location = New System.Drawing.Point(448, 12)
Me.btnFile.Name = "btnFile"
Me.btnFile.Size = New System.Drawing.Size(75, 23)
Me.btnFile.TabIndex = 10
Me.btnFile.Text = "Browse..."
Me.btnFile.UseVisualStyleBackColor = True
'Me.btnFile.Click += New System.EventHandler(Me.btnFile_Click)
'
' OpenFileDialog1
'
Me.OpenFileDialog1.FileName = "OpenFileDialog1"
'
' txtFileName
'
Me.txtFileName.Location = New System.Drawing.Point(12, 12)
Me.txtFileName.Name = "txtFileName"
Me.txtFileName.Size = New System.Drawing.Size(430, 20)
Me.txtFileName.TabIndex = 9
'
' groupBox1
'
Me.groupBox1.Controls.Add(Me.label3)
Me.groupBox1.Controls.Add(Me.txtStreamName)
Me.groupBox1.Controls.Add(Me.label1)
Me.groupBox1.Controls.Add(Me.txtStoreName)
Me.groupBox1.Controls.Add(Me.rtbStore)
Me.groupBox1.Location = New System.Drawing.Point(13, 39)
Me.groupBox1.Name = "groupBox1"
Me.groupBox1.Size = New System.Drawing.Size(210, 180)
Me.groupBox1.TabIndex = 18
Me.groupBox1.TabStop = False
Me.groupBox1.Text = "Store:"
'
' txtStoreName
'
Me.txtStoreName.Location = New System.Drawing.Point(88, 17)
Me.txtStoreName.Name = "txtStoreName"
Me.txtStoreName.Size = New System.Drawing.Size(116, 20)
Me.txtStoreName.TabIndex = 15
'
' label1
'
Me.label1.AutoSize = True
Me.label1.Location = New System.Drawing.Point(7, 20)
Me.label1.Name = "label1"
Me.label1.Size = New System.Drawing.Size(63, 13)
Me.label1.TabIndex = 16
Me.label1.Text = "StoreName:"
'
' label3
'
Me.label3.AutoSize = True

```

```

Me.label3.Location = New System.Drawing.Point(7, 53)
Me.label3.Name = "label3"
Me.label3.Size = New System.Drawing.Size(75, 13)
Me.label3.TabIndex = 18
Me.label3.Text = "StorageName:"
'
' txtStreamName
'
Me.txtStreamName.Location = New System.Drawing.Point(88, 50)
Me.txtStreamName.Name = "txtStreamName"
Me.txtStreamName.Size = New System.Drawing.Size(116, 20)
Me.txtStreamName.TabIndex = 17
'
' groupBox2
'
Me.groupBox2.Controls.Add(Me.rtbStorage)
Me.groupBox2.Controls.Add(Me.label5)
Me.groupBox2.Controls.Add(Me.txtStorageName)
Me.groupBox2.Location = New System.Drawing.Point(232, 39)
Me.groupBox2.Name = "groupBox2"
Me.groupBox2.Size = New System.Drawing.Size(210, 180)
Me.groupBox2.TabIndex = 19
Me.groupBox2.TabStop = False
Me.groupBox2.Text = "Storage:"
'
' label5
'
Me.label5.AutoSize = True
Me.label5.Location = New System.Drawing.Point(7, 20)
Me.label5.Name = "label5"
Me.label5.Size = New System.Drawing.Size(75, 13)
Me.label5.TabIndex = 16
Me.label5.Text = "StorageName:"
'
' txtStorageName
'
Me.txtStorageName.Location = New System.Drawing.Point(88, 17)
Me.txtStorageName.Name = "txtStorageName"
Me.txtStorageName.Size = New System.Drawing.Size(116, 20)
Me.txtStorageName.TabIndex = 15
'
' Form1
'
Me.AutoScaleDimensions = New System.Drawing.SizeF(6.0F, 13.0F)
Me.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font
Me.ClientSize = New System.Drawing.Size(536, 231)
Me.Controls.Add(Me.btnExit)
Me.Controls.Add(Me.btnSave)
Me.Controls.Add(Me.btnOpen)
Me.Controls.Add(Me.btnFile)
Me.Controls.Add(Me.txtFileName)
Me.Controls.Add(Me.groupBox1)
Me.Controls.Add(Me.groupBox2)
Me.MaximumSize = New System.Drawing.Size(552, 269)
Me.MinimumSize = New System.Drawing.Size(552, 269)
Me.Name = "Form1"
Me.Text = "DocManager Test"
Me.groupBox1.ResumeLayout(False)
Me.groupBox1.PerformLayout()
Me.groupBox2.ResumeLayout(False)
Me.groupBox2.PerformLayout()
Me.ResumeLayout(False)
Me.PerformLayout()

```

End Sub

```

Friend WithEvents rtbStorage As System.Windows.Forms.RichTextBox
Friend WithEvents rtbStore As System.Windows.Forms.RichTextBox
Friend WithEvents btnExit As System.Windows.Forms.Button
Friend WithEvents btnSave As System.Windows.Forms.Button
Friend WithEvents btnOpen As System.Windows.Forms.Button
Friend WithEvents btnFile As System.Windows.Forms.Button
Friend WithEvents OpenFileDialog1 As System.Windows.Forms.OpenFileDialog
Friend WithEvents txtFileName As System.Windows.Forms.TextBox
Private groupBox1 As System.Windows.Forms.GroupBox
Private label3 As System.Windows.Forms.Label
Private txtStreamName As System.Windows.Forms.TextBox
Private label1 As System.Windows.Forms.Label
Private txtStoreName As System.Windows.Forms.TextBox
Private groupBox2 As System.Windows.Forms.GroupBox
Private label5 As System.Windows.Forms.Label
Private txtStorageName As System.Windows.Forms.TextBox

```

End Class

Get BOM Tables Example (C#)

This example shows how to get the BOM tables in a drawing document.

```
//-----
// Preconditions:
// 1. Create a console application using Visual C#.
// 2. Verify that all output is redirected to the Immediate window. (In
//    Visual Studio 2015, click Tools > Options > Debugging > General >
//    Redirect all Output Window text to the Immediate Window).
// 3. Click Project > Add Reference > install_dir/api/redirect/
//    SolidWorks.Interop.swdocumentmgr.dll > OK.
// 4. Substitute your license key for your_license_key in the code.
// 5. Copy the specified drawing document to another file name.
// 6. Open the specified drawing document in SOLIDWORKS.
// 7. Ensure that both Bills of Materials are not hidden.
// 8. Save and rebuild the drawing.
// 9. Close SOLIDWORKS.
//
// Postconditions: Prints to the Immediate window:
// * Document:
// * Path and filename
// * Last saved date
// * For each BOM table that is not hidden in the document:
// * Name
// * Whether hidden
// * Whether to display components with multiple configurations as one item
// * Part configuration grouping setting
// * Sheet name
// * Number of views on the sheet
// * Name of the custom property view for the sheet
//
// NOTE: Restore the original drawing document, as it is used elsewhere.
//-----

using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

namespace ConsoleApplication2
{
    class Module1
    {
        static void Main()
        {
            SwDMClassFactory dmClassFact = default(SwDMClassFactory);
            SwDMApplication3 dmDocMgr = default(SwDMApplication3);
            ISwDMDocument15 dmDoc = default(ISwDMDocument15);
            ISwDMTable5 dmTable = default(ISwDMTable5);
            ISwMSheet4 dmSheet = default(ISwMSheet4);
            ISwDMView custPropView = default(ISwDMView);
            SwDmDocumentOpenError dmError = default(SwDmDocumentOpenError);
            Array arrViews = null;
            string nameDrawing = "";

            dmClassFact = new SwDMClassFactory();
            dmDocMgr = (SwDMApplication3)dmClassFact.GetApplication("your_license_key"); //Do not distribute
            nameDrawing = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2017\\introsw\\cabinet_bath.slddrw";

            //Get the SOLIDWORKS drawing document
            dmDoc = (ISwDMDocument15)dmDocMgr.GetDocument(nameDrawing, SwDmDocumentType.swDmDocumentDrawing, false, out dmError);
            Debug.Print("Document's full name: " + dmDoc.FullName);
            Debug.Print("Date document last saved: " + dmDoc.LastSavedDate);

            // Get the names of the BOM tables that are not hidden in the SOLIDWORKS drawing document
            string[] tableNames = null;
            tableNames = (string[])dmDoc.GetTableNames(SwDmTableType.swDmTableTypeBOM);
            int i = 0;
            if ((tableNames != null))
            {
                for (i = 0; i <= tableNames.Length - 1; i++)
                {
                    dmTable = (SwDMTable5)dmDoc.GetTable(tableNames[i]);
                    Debug.Print(" BOM table name: " + dmTable.Name);
                    Debug.Print(" Is BOM table hidden? " + dmTable.Hidden);
                    Debug.Print(" If BOM table contains components with multiple configurations, display as one item? " + dmTable.
                        " + dmTable.PartConfigurationGrouping);
                    dmSheet = (SwMSheet4)dmTable.Sheet;
                    if (dmSheet != null)
                    {
                        custPropView = dmSheet.CustomPropertyView;
                        arrViews = dmSheet.GetViews();
                        Debug.Print(" On sheet: " + dmSheet.Name);
                        Debug.Print(" Number of views on sheet: " + arrViews.Length);
                        Debug.Print(" Custom property view: " + custPropView.Name);
                    }
                }
            }
        }
    }
}
```

} } } }

Get BOM Tables Example (VB.NET)

This example shows how to get the BOM tables in a drawing document.

```

'-----
' Preconditions:
' 1. Create a console application using Visual Basic.
' 2. Verify that all output is redirected to the Immediate window. (In
' Visual Studio 2015, click Tools > Options > Debugging > General >
' Redirect all Output Window text to the Immediate Window).
' 3. Click Project > Add Reference > install_dir/api/redist/
' SolidWorks.Interop.swdocumentmgr.dll > OK.
' 4. Substitute your license key for your_license_key in the code.
' 5. Copy the specified drawing document to another file name.
' 6. Open the specified drawing document in SOLIDWORKS.
' 7. Ensure that both Bills of Materials are not hidden.
' 8. Save and rebuild the drawing.
' 9. Close SOLIDWORKS.
'
' Postconditions: Prints to the Immediate window:
' * Document:
'   * Path and file name
'   * Last saved date
' * For each BOM table that is not hidden in the document:
'   * Name
'   * Whether hidden
'   * Whether to display components with multiple configurations as one item
'   * Part configuration grouping setting
'   * Sheet name
'   * Number of views on the sheet
'   * Name of the custom property view for the sheet
'
' NOTE: Restore the original drawing document, as it is used elsewhere.
'-----

```

```

Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics

```

```
Module Module1
```

```

    Dim dmClassFact As SwDMClassFactory
    Dim dmDocMgr As SwDMApplication3
    Dim dmDoc As ISwMDocument15
    Dim dmTable As ISwDMTable5
    Dim dmSheet As ISwDMSheet4
    Dim custPropView As ISwDMView
    Dim dmError As SwDmDocumentOpenError
    Dim arrviews As Array
    Dim nameDrawing As String

```

```
Sub Main()
```

```

    dmClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
    dmDocMgr = dmClassFact.GetApplication("your_license_key") 'Do not distribute
    nameDrawing = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\cabinet_bath.slddrw"

```

```

' Get the SOLIDWORKS drawing document
    dmDoc = dmDocMgr.GetDocument(nameDrawing, SwDmDocumentType.swDmDocumentDrawing, False, dmError)
    Debug.Print("Document's full name: " & dmDoc.FullName)
    Debug.Print("Date document last saved: " & dmDoc.LastSavedDate)

```

```
' Get the names of the BOM tables that are not hidden in the SOLIDWORKS drawing document
```

```

Dim tableNames As Object
tableNames = dmDoc.GetTableNames(SwDmTableType.swDmTableTypeBOM)

```

```
Dim i As Integer
```

```
If Not IsNothing(tableNames) Then
```

```
    For i = 0 To UBound(tableNames)
```

```
        dmTable = dmDoc.GetTable(tableNames(i))
```

```
        Debug.Print(" BOM table name: " & dmTable.Name)
```

```
        Debug.Print(" Is BOM table hidden? " & dmTable.Hidden)
```

```
        Debug.Print(" If BOM table contains components with multiple configurations, display as one item? " & dmTable.DisplayAsOneItem)
```

```
        Debug.Print(" Part configuration grouping as defined in swDMPartConfigurationGrouping:
```

```
" & dmTable.PartConfigurationGrouping)
```

```
        dmSheet = dmTable.Sheet
```

```
        If Not IsNothing(dmSheet) Then
```

```
            custPropView = dmSheet.CustomPropertyView
```

```
            arrviews = dmSheet.GetViews
```

```
            Debug.Print(" On sheet: " & dmSheet.Name)
```

```
            Debug.Print(" Number of views on sheet: " & arrviews.Length)
```

```
            Debug.Print(" Custom property view: " & custPropView.Name)
```

```
        End If
```

```
    Next
```

```
End If
```

```
End Sub
```

```
End Module
```

Get Component Reference Example (C#)

This example shows how to replace a component with a different component.

```

'-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
//    topic and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a VB.NET console application
//    in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click:
//        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
//    e. Click Add.
//    f. Click Close.
// 4. Ensure that the model to open exists.
// 5. Substitute your_license_key with your SOLIDWORKS Document
//    Manager license key.
// 6. Open the Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The first instance of the shaft washer component is
'    replaced with a lock washer in the document.
'
' NOTE: The specified file should be backed up before running this macro,
' as it is used elsewhere. The macro uses a replacement that does not
' properly fit in the model.
'-----

using System;
using System.Diagnostics;
using SolidWorks.Interop.swdocumentmgr;
namespace ConsoleApplication2
{
    class Program
    {
        static void Main(string[] args)
        {
            SwDMClassFactory swCf;
            swCf = new SwDMClassFactory();
            SwDMAplication swDocMgr;
            swDocMgr = (SwDMAplication)swCf.GetApplication("your_license_key");
            SwDMDocument12 swDoc12;
            SwDmDocumentOpenError res;
            SwDmDocumentType dt;
            dt = SwDmDocumentType.swDmDocumentAssembly;
            string filename;
            Debug.Print("Opening an assembly...");

            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\98food
processor.sldasm";
            swDoc12 = swDocMgr.GetDocument(filename, dt, false, out res) as SwDMDocument12;

            if (swDoc12 == null | (res != SwDmDocumentOpenError.swDmDocumentOpenErrorNone))
            {
                Debug.Print("Error opening file...");
                return;
            }

            Debug.Print("Getting the active configuration...");
            SwDMConfiguration8 activeConfig;
            SwDMConfigurationMgr configMgr;
            configMgr = swDoc12.ConfigurationManager;
            configMgr.GetConfigurationNames();
            activeConfig = configMgr.GetConfigurationByName(configMgr.GetActiveConfigurationName()) as SwDMConfiguration8;
            if (activeConfig == null)
            {
                Debug.Print("Error getting the active configuration...");
                return;
            }

            Debug.Print("Getting the components of the active configuration...");
            object[] vComponents;
            vComponents = (object[])activeConfig.GetComponents();
            SwDMComponent6 swDmComponent;

            int i;
            for (i = 0; i < vComponents.Length; i++)
            {
                swDmComponent = (SwDMComponent6)vComponents[i];
                if (swDmComponent.Name == "shaft washer")
                {
                    bool bResult = swDmComponent.Replace("C:\\Program Files\\SOLIDWORKS

```

```
Corp\\SOLIDWORKS\\samples\\tutorial\\smartcomponents\\lockwasher.sldprt", "Default", false);
    Debug.Print("Replacing shaft washer with lock washer...");
    Debug.Print(bResult.ToString());
    break;
}
}

swDoc12.Save();
swDoc12.CloseDoc();
}
}
```

Get Component Reference Example (VB.NET)

This example shows how to replace a component with a different component.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started
'    topic and ensure that the required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
'    in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
'    a. Right-click the solution in Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click:
'        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
'    e. Click Add.
'    f. Click Close.
' 4. Ensure that the model to open exists.
' 5. Substitute your_license_key with your SOLIDWORKS Document
'    Manager license key.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The first instance of the shaft washer component is
'    replaced with a lock washer in the document.
'
' NOTE: The specified file should be backed up before running this macro,
' as it is used elsewhere. The macro uses a replacement that does not
' properly fit in the model.
'-----

```

Imports System.Diagnostics

Imports SolidWorks.Interop.swdocumentmgr

Module Module1

Sub Main()

```

    Dim swCf As SwDMClassFactory
    swCf = New SwDMClassFactory()
    Dim swDocMgr As SwDMAApplication
    swDocMgr = DirectCast(swCf.GetApplication("your_license_key"), SwDMAApplication)
    Dim swDoc12 As SwDMDocument12
    Dim res As SwDmDocumentOpenError
    Dim dt As SwDmDocumentType
    dt = SwDmDocumentType.SwDmDocumentAssembly
    Dim filename As String

```

```

    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food processor.sldasm"
    swDoc12 = TryCast(swDocMgr.GetDocument(filename, dt, False, res), SwDMDocument12)

```

```

    If swDoc12 Is Nothing Or (res <> SwDmDocumentOpenError.swDmDocumentOpenErrorNone) Then
        Debug.Print("Error opening file...")
        Return
    End If

```

```

    Debug.Print("Getting the active configuration...")
    Dim activeConfig As SwDMConfiguration8
    Dim configMgr As SwDMConfigurationMgr
    configMgr = swDoc12.ConfigurationManager
    configMgr.GetConfigurationNames()
    activeConfig = TryCast(configMgr.GetConfigurationByName(configMgr.GetActiveConfigurationName()), SwDMConfiguration8)
    If activeConfig Is Nothing Then
        Debug.Print("Error getting the active configuration...")
        Return
    End If

```

```

    Debug.Print("Getting the components of the active configuration...")
    Dim vComponents As Object()
    vComponents = DirectCast(activeConfig.GetComponents(), Object())
    Dim swDmComponent As SwDMComponent6

```

```

    Dim i As Integer
    For i = 0 To vComponents.Length - 1
        swDmComponent = DirectCast(vComponents(i), SwDMComponent6)
        If swDmComponent.Name = "shaft washer" Then
            Dim bResult As Boolean = swDmComponent.Replace("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\smartcomponents\lockwasher.sldprt", "Default", False)
            Debug.Print("Replacing shaft washer with lock washer...")
            Debug.Print(bResult.ToString())
            Exit For
        End If
    Next

```

```

    swDoc12.Save()
    swDoc12.CloseDoc()

```

End Sub

End Module

Get Configuration-Specific Custom Properties for Components (C#)

This example shows how to get custom properties for assembly components.

```
//-----  
// Preconditions:  
// 1. Read the SOLIDWORKS Document Manager API Getting Started  
//    topic and ensure that the required DLLs are registered.  
// 2. Copy and paste this code into a C# console application  
//    in Microsoft Visual Studio.  
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:  
//    a. Right-click the solution in Solution Explorer.  
//    b. Click Add Reference.  
//    c. Click Browse.  
//    d. Click:  
//        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll  
//    e. Click Add.  
//    f. Click Close.  
// 4. Ensure that the model to open exists.  
// 5. Substitute your_license_key with your SOLIDWORKS Document  
//    Manager license key.  
// 6. Open the Immediate window.  
//  
// Postconditions:  
// 1.  
// 2.  
//  
// NOTE: Because the model is used elsewhere, do not save changes.  
//-----
```